

Part 1_2_3_4 (Strictly Video Friendly)

DevOps All-Rounder Guide

From Zero to Production: Linux → Docker → CI/CD → AWS
→ Kubernetes

How to Use This Guide

- **"Logic Questions"** in Interview Edge sections = explain to your interviewer (the *why*)
- **Code blocks** = show to your teammates (the *how*)
- Quick-Reference Tables = your cheat sheet during interviews and on the job
- MERN examples are woven throughout every phase to show real-world context

PHASE 1: THE ENVIRONMENT

Linux, Shell Scripting, and Version Control

1.1 — What is DevOps? (The Foundation)

DevOps is a set of practices that **combines software development (Dev) and IT operations (Ops)** into a unified, automated workflow. Its primary goal is to shorten the development lifecycle while delivering features, fixes, and updates frequently, reliably, and safely.

Development + Operations = DevOps

Before DevOps, developers wrote code and "threw it over the wall" to operations teams to deploy. This led to:

- Manual, error-prone deployments
- "Works on my machine" syndrome
- Slow feedback cycles

- Breaking production unexpectedly

Key Benefits:

- **Faster builds** — Automated pipelines replace manual steps
 - **Safer deployments** — Automated testing catches bugs before production
 - **Easier maintainability** — Infrastructure-as-code means reproducible environments
-

The 7 Pillars of the DevOps Lifecycle

```
Plan    → Define requirements, roadmap, and scope with
product/engineering teams
↓
Develop → Write source code, set up repositories
↓
Build    → CI tools automatically compile and package code
↓
Test     → Automated tests validate functionality and quality
↓
Release  → Finalize builds and prepare for production promotion
↓
Deploy   → CD pushes the application to servers accessible by users
↓
Monitor  → Track performance and capture real-time data to resolve
issues
```

This guide follows exactly this lifecycle — Phase 1 covers Plan/Develop, Phase 2 covers Build, Phase 3 covers Test/Release/Deploy, and Phase 4 covers the infrastructure that makes it all scale.

1.2 — Linux in DevOps

Why Linux?

Linux is the backbone of DevOps because:

- **All major DevOps tools** (Docker, Kubernetes, Git, Jenkins) are optimized for Linux

- **Cloud servers** (AWS, Azure, GCP) run Linux by default
- **Stability, security, and performance** — Linux servers run for years without reboots
- **Container base images** — Docker containers run Linux kernels

Accessing Linux on Windows (WSL)

```
wsl --install          # Installs Ubuntu by default
wsl --list --verbose   # List installed distros
wsl                   # Start Ubuntu shell
```

Linux Quick-Reference Table

Command	Use Case
<code>pwd</code>	Shows your current working directory path — essential for knowing where you are before running scripts
<code>ls</code>	Lists files and directories in the current folder
<code>ls -l</code>	Lists files with detailed info: permissions, size, owner, timestamp
<code>ls -a</code>	Lists all files including hidden ones (e.g., <code>.env</code> , <code>.gitignore</code>)
<code>cd <dir></code>	Changes the current directory
<code>cd ~</code>	Go to your home directory
<code>cd /</code>	Moves to the root directory of the filesystem
<code>touch <file></code>	Creates an empty file (or updates a file's timestamp if it exists)
<code>rm <file></code>	Deletes a file permanently — no recycle bin
<code>rm -r <dir></code>	Deletes a directory recursively, including all contents — always use this for non-empty directories
<code>mkdir <dir></code>	Creates a new directory
<code>rmdir <dir></code>	Deletes an empty directory only
<code>cp file1 file2</code>	Copies a file to a new location or name
<code>mv file1 newloc/</code>	Moves or renames a file

Command	Use Case
<code>find <path> -name "<file>"</code>	Searches for files by name in an entire directory hierarchy — useful for locating config files
<code>grep "<text>" <file></code>	Searches for specific text inside a file — used to scan logs, find environment variables, etc.
<code>vi <file></code>	Opens a file in Vim editor for in-place editing on servers
<code>nano <file></code>	Opens a file in the nano editor — simpler than Vim for beginners
<code>history</code>	Shows previously executed commands — useful for auditing what was run on a server
<code>chmod +x <file></code>	Makes a file executable — required before running a shell script with <code>./script.sh</code>

1.3 — Shell Scripting

A shell script is a text file containing a series of commands executed by the Linux shell. Its primary purpose is **automation** — replacing repetitive manual tasks with a single executable file.

The 4-Step Workflow

```
# 1. Create the file
```

```
nano script.sh
```

```
# 2. Write your commands (with shebang line at top)
```

```
#!/bin/bash
```

```
echo "Hello World"
```

```
# 3. Save and exit: Ctrl+O → Enter → Ctrl+X
```

```
# 4. Make it executable
```

```
chmod +x script.sh
```

```
# 5. Run it
```

```
./script.sh
```

Why `#!/bin/bash` ? The "shebang" line tells the OS which interpreter to use to execute the script. Without it, the shell doesn't know what language the

script is written in.

Variables

```
#!/bin/bash
name="Alice"
echo "Hello $name"
```

Conditional Statements

```
#!/bin/bash
number=6

if [ $number -gt 5 ]; then
    echo "Num greater than 5"
else
    echo "Nah, It's smaller"
fi
```

Running it:

```
$ chmod +x conditional.sh
$ ./conditional.sh
Num greater than 5
```

Common Comparison Operators

Operator	Meaning
-gt	Greater than
-lt	Less than
-eq	Equal to
-ne	Not equal to
-ge	Greater than or equal
-le	Less than or equal

MERN Implementation: Shell Script to Start the App

In a MERN project, before CI/CD pipelines take over, a shell script can automate local startup:

```
#!/bin/bash
echo "Starting MERN app..."

# Start backend in the background (&)
cd server && npm start &

# Start frontend in the foreground
cd ../client && npm run dev
```

DevOps Use Cases for Shell Scripts:

- Automating build/test commands before a CI pipeline is set up
- Setting environment variables at startup
- Deploy scripts (initial server setup before Jenkins/GitHub Actions)
- Server provisioning (installing Docker, Git, etc. on a fresh EC2 instance)

1.4 — Git & GitHub

Git vs. GitHub

	Git	GitHub
What it is	A distributed version control system (VCS) that runs locally	A cloud-based hosting service built on top of Git
Lives	On your local machine	On the internet (cloud)
Purpose	Tracks changes in code, manages history	Enables collaboration, pull requests, CI/CD integration
Key fact	Every developer has a full copy of the repo and its entire history	Acts as the "source of truth" remote repository

Git Quick-Reference Table

Command	Use Case
<code>git init</code>	Initializes a new local repository in the current directory — the first command in any new project
<code>git status</code>	Checks the current state of tracked/untracked files — run this before any add/commit
<code>git add <file></code>	Moves a specific file's changes to the staging area (prepares for commit)
<code>git add .</code>	Stages all changed files — use carefully to avoid committing unwanted files
<code>git commit -m "message"</code>	Creates a permanent snapshot of staged changes with a descriptive message
<code>git log</code>	Shows the commit history — each commit has a hash, author, date, and message
<code>git push origin <branch></code>	Uploads local commits to the remote GitHub repository
<code>git pull</code>	Fetches and merges remote changes into your current branch
<code>git remote add origin <url></code>	Links the local repo to a remote GitHub URL
<code>git remote -v</code>	Verifies configured remote connections
<code>git branch <name></code>	Creates a new branch
<code>git checkout <branch></code>	Switches to an existing branch
<code>git checkout -b <branch></code>	Creates AND switches to a new branch in one command
<code>git switch <branch></code>	Modern alternative to <code>git checkout</code> for switching branches
<code>git merge <branch></code>	Merges another branch into the current one
<code>git branch -M main</code>	Renames the current branch to <code>main</code>
<code>git push -u origin main</code>	Pushes and sets upstream tracking for a branch (only needed once)

Practical Implementation

1. Initialization & First Push

```
# Initialize repo
git init

# Stage and commit
git add .
git commit -m "Initial commit: MERN project setup"

# Connect to GitHub and push
git remote add origin https://github.com/user/repo.git
git branch -M main
git push -u origin main
```

2. Branching Strategy (Critical for Team Development)

Developers **never push directly to** `main`. The workflow is:

```
# Create a feature branch
git checkout -b feature/add-login

# ... make changes, add, commit ...
git add .
git commit -m "feat: add user login endpoint"

# Push feature branch to GitHub
git push origin feature/add-login

# Then raise a Pull Request on GitHub to merge into main/dev
```

Why branching matters in DevOps:

- Keeps `main` stable and deployment-ready at all times
- Pull Requests (PRs) trigger automated CI checks before code merges
- Multiple developers can work in parallel without conflicts

3. .gitignore — What NOT to Push

```
node_modules/
.env
dist/
```



```
.DS_Store
*.log
```

Critical: Never push `.env` files. They contain secrets (database URIs, API keys) that would be exposed to anyone who can see the repo.

1.5 — Environment Management

Why It Matters

You must **never hardcode sensitive credentials** (database URLs, API keys, JWT secrets) directly in your source code. If pushed to a public repository, this exposes your entire infrastructure.

The Three Environment Tiers

```
GitHub → Dev (Localhost) → Staging (test server) → Production (live)
```

Environment	Purpose	Who Uses It
Development (Dev)	Initial coding and local testing	Developers locally
Staging	Mirrors production exactly for final testing	QA, Product managers
Production (Prod)	Live environment with real users	End users

Using .env Files

```
# server/.env (development)
PORT=5000
MONGO_URI=mongodb://localhost:27017/dev_db
NODE_ENV=development

# server/.env.prod (production)
PORT=80
MONGO_URI=mongodb+srv://user:pass@cluster.mongodb.net/prod
NODE_ENV=production
```

Always add `.env` to `.gitignore`:

```
echo ".env" >> .gitignore
```

MERN Implementation

Backend (Node + Express):

```
import dotenv from 'dotenv';
dotenv.config();

const PORT = process.env.PORT;
const MONGO_URI = process.env.MONGO_URI;
```

Frontend (React + Vite):

```
# client/.env.development
VITE_API_URL=http://localhost:5000/api

# client/.env.production
VITE_API_URL=https://your-prod-domain.com/api
```

Switching automatically:

```
npm run dev      # Vite uses .env.development
npm run build    # Vite uses .env.production
```

Important Vite Rule: Variables prefixed with `VITE_` are **embedded at build time** into the static JS bundle. This means if you change an environment variable after the build, the JS files won't pick it up — you must rebuild. This has direct implications for Docker and Kubernetes deployments (covered in later phases).

PHASE 2: CONTAINERIZATION

Docker, Volumes, and Networking

2.1 — Docker Architecture: Deep Dive

What & Why

What Docker is: A tool that packages an application with everything it needs — code, libraries, runtime, and configurations — into a single, lightweight **container**. This ensures the app runs identically in any environment: your laptop, a teammate's machine, or a cloud server.

The Old Way (The Problem): Before containers, deploying an app meant:

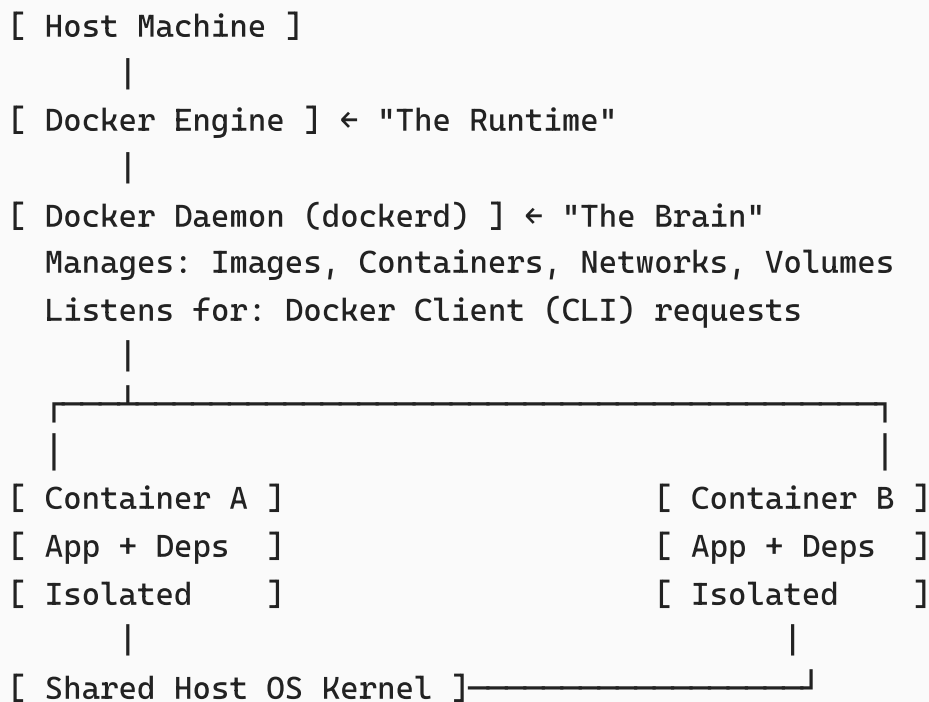
1. Manually installing the right version of Node.js, Python, etc. on each server
2. Managing OS-level dependencies that differed between developer laptops and servers
3. "It works on my machine!" — apps behaving differently in different environments
4. Full Virtual Machines (VMs) to solve isolation — but VMs are heavy (GBs), slow to start, and wasteful

What Docker Solves (What VMs Couldn't Do Efficiently):

Problem	VM Approach	Docker Approach
Isolation	Run entire OS inside a VM	Lightweight namespace/cgroup isolation
Startup time	Minutes	Milliseconds
Size	GBs (includes full OS)	MBs (shares host OS kernel)
Portability	VMs are tied to hypervisors	Containers run anywhere Docker runs
Density	5-10 VMs on a server	Hundreds of containers on a server

The Key Architectural Insight: Containers **share the host OS kernel** but remain logically isolated from each other using Linux namespaces and cgroups. They don't need a full OS — they just need the application and its dependencies.

The Architecture



Key Components:

Docker Daemon (`dockerd`): The central background service on the host OS. It is the "brain" of Docker, responsible for:

- Managing the lifecycle of images, containers, networks, and volumes
- Listening for requests from the Docker CLI
- Executing processes (e.g., `docker run`) and streaming output back to your terminal

Image: A read-only "blueprint" (like a zip file) containing your application code and environment setup. Images are layered — each Dockerfile instruction adds a new layer. Images are pulled from Docker Hub (the public image registry).

Container: The actual **running instance** of an image. A container is an image that has been started and given a writable layer on top.

Docker Engine: The system that runs containers, consisting of the daemon and client.

The Dockerfile: Building Your Image

A Dockerfile is a text document with instructions to build a container image.

Backend Dockerfile (Node.js):

```
# server/Dockerfile

# 1. Base image: lightweight Linux with Node.js 18 pre-installed
FROM node:18-alpine

# 2. Set the working directory inside the container
# All subsequent commands run relative to this path
WORKDIR /app

# 3. Copy ONLY package.json first (for layer caching)
# This is a critical optimization: if package.json hasn't
# changed,
# Docker reuses the cached npm install layer, making rebuilds
# much faster
COPY package*.json ./

# 4. Install dependencies (runs DURING BUILD, not at runtime)
RUN npm install

# 5. Copy the rest of the application code
COPY . .

# 6. Document which port the container listens on (metadata only)
EXPOSE 5000

# 7. The command that runs WHEN THE CONTAINER STARTS
# Unlike RUN (which runs at build time), CMD runs at runtime
CMD ["node", "index.js"]
```

Frontend Dockerfile (React/Vite):

```
# client/Dockerfile

FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 5173
CMD ["npm", "run", "dev"]
```

Key Dockerfile Instruction Distinction:

Instruction	When it runs	Purpose
RUN	At build time	Install dependencies, compile code
CMD	At runtime (container start)	Start the application
COPY	At build time	Copy files from host into image
EXPOSE	Documentation only	Declare the intended container port

Docker Quick-Reference Table

Command	Use Case
<code>docker --version</code>	Verify Docker is installed
<code>docker run hello-world</code>	Basic sanity check — pulls and runs a test container
<code>docker run -d -p 8080:80 nginx</code>	Run nginx web server in detached mode, mapping host port 8080 to container port 80
<code>docker run -d -p <host>:<container> <image></code>	Standard way to start a container: detached mode with port mapping
<code>docker build -t <image_name> .</code>	Build an image from the Dockerfile in the current directory, tagged with a name
<code>docker ps</code>	List all currently running containers
<code>docker ps -a</code>	List all containers including stopped ones
<code>docker stop <id></code>	Gracefully stop a running container (sends SIGTERM, waits, then SIGKILL)
<code>docker rm <id></code>	Remove a stopped container
<code>docker rmi <image></code>	Remove a local image
<code>docker logs <id></code>	View the stdout/stderr output of a container
<code>docker exec -it <id> bash</code>	Open an interactive bash shell inside a running container
<code>docker inspect <id></code>	Show low-level JSON config of a container (network, volumes, env vars)
<code>docker images</code>	List all local images
<code>docker pull <image></code>	Pull an image from Docker Hub without running it

Command	Use Case
<code>docker push <image></code>	Push a tagged image to a registry
<code>docker tag <source> <target></code>	Create a new tag for an existing image
<code>docker login</code>	Authenticate to Docker Hub
<code>docker volume ls</code>	List all Docker volumes
<code>docker volume create <name></code>	Create a named volume
<code>docker volume inspect <name></code>	Show detailed info about a volume
<code>docker network ls</code>	List all Docker networks
<code>docker network create <name></code>	Create a custom user-defined bridge network
<code>docker network inspect <name></code>	Show detailed info about a network
<code>docker image prune -f</code>	Remove all dangling (unused) images — clean up disk space

2.2 — Docker Compose

Why Docker Compose is Needed

The Problem: A MERN stack has at least 3 services: MongoDB, Node.js backend, and React frontend. Manually starting each container with `docker run`, managing their ports, environment variables, and connecting them to the same network is tedious and error-prone.

The Analogy: Docker CLI is a single musician. Docker Compose is an entire orchestra — it coordinates all instruments (services) from one conductor (the `docker-compose.yml` file).

docker-compose.yml (MERN Example)

```
version: '3.8'

services:
  mongo:
    image: mongo:6.0
```

```

    container_name: mongo
    restart: unless-stopped
    volumes:
      - mongo-data:/data/db          # Named volume for data
persistence
    networks:
      - mern-net

backend:
  build:
    context: ./server
    dockerfile: Dockerfile
  container_name: backend
  restart: unless-stopped
  env_file:
    - ./server/.env                # Load env vars from file
  ports:
    - "5000:5000"
  depends_on:
    - mongo                        # Wait for mongo before starting
  networks:
    - mern-net

frontend:
  build:
    context: ./client
    dockerfile: Dockerfile
    args:
      VITE_API_URL: http://localhost:5000/api  # Build-time arg
for Vite
  container_name: frontend
  restart: unless-stopped
  ports:
    - "3000:5173"
  depends_on:
    - backend
  networks:
    - mern-net

volumes:
  mongo-data:          # Named volume definition (persists across
restarts)

networks:

```



```
mern-net:
  driver: bridge    # Custom user-defined bridge network
```

Docker Compose Quick-Reference Table

Command	Use Case
<code>docker compose up --build</code>	Build images and start all services — use on first run or after code changes
<code>docker compose up -d --build</code>	Same as above but runs in detached (background) mode
<code>docker compose down</code>	Stop and remove all containers and networks for the project
<code>docker compose ps</code>	List all running services and their status
<code>docker compose logs</code>	View combined logs from all services
<code>docker compose logs <service></code>	View logs for a specific service (e.g., <code>docker compose logs backend</code>)
<code>docker compose logs -f</code>	Follow (tail) logs in real time
<code>docker compose stop</code>	Stop containers without removing them
<code>docker compose start</code>	Start previously stopped containers
<code>docker compose build</code>	Build or rebuild images without starting them

FAQ: Why Do Individual Dockerfiles Still Exist If Docker Compose Manages Everything?

Dockerfiles are the fundamental blueprints that define *how to build* a specific image — installing dependencies, copying code, setting environments. They operate at the image level.

Docker Compose is an orchestrator. Its job is not to build images from scratch, but to manage the *lifecycle* of multiple containers simultaneously — defining services, networking, dependencies, and volumes. When Docker Compose performs a build, it *references* the individual Dockerfiles for each service to actually create the images.

Think of it this way: Dockerfiles are recipes. Docker Compose is the restaurant kitchen manager that decides which recipe to cook, when to cook it, and how to serve the meal.

2.3 — Docker Volumes: Persistence Deep Dive

The Core Problem

Containers are **ephemeral by design**. When a container is removed, its entire internal filesystem is wiped. For a MongoDB container, this means your entire database disappears every time you stop and recreate the container.

Without volumes:

Container A (running) → writes to /data/db → Container A deleted → DATA GONE

With volumes:

Container A (running) → writes to /data/db → mapped to named volume "mongo-data"

Container A deleted → DATA STILL EXISTS in volume

Container B created (same volume) → DATA ACCESSIBLE in new container

Volume Types

Named Volumes (Recommended for Production):

```
# Create a named volume (Docker manages the storage location)
```

```
docker volume create my-app-data
```

```
# Use in a container
```

```
docker run -v my-app-data:/data/db mongo:6.0
```

Docker stores the data at `/var/lib/docker/volumes/my-app-data/_data` — fully managed by Docker.

Anonymous Volumes:

```
# Docker creates a random name
```

```
docker run -v /data/db mongo:6.0
```

Similar to named volumes but hard to reference by name later — avoid in production.

Bind Mounts:

```
# Mount a specific host directory into the container
docker run -v /host/path:/container/path my-app
```

Directly links a host filesystem path to a container path. Commonly used in development for hot-reloading.

Hands-On Persistence Demo

```
# Step 1: Create a named volume
docker volume create demo-vol
docker volume ls
docker volume inspect demo-vol

# Step 2: Run MongoDB with the volume
docker run -d --name demo-mongo -v demo-vol:/data/db mongo:6.0

# Step 3: Add some data to the database
docker exec -it demo-mongo mongosh --eval "
  use testdb;
  db.demos.insertOne({ name: 'docker-persist-demo', ts: new Date()
})
"

# Step 4: Verify data exists
docker exec -it demo-mongo mongosh --eval "
  use testdb;
  db.demos.find().pretty()
"

# Step 5: Delete the container (the scary part!)
docker rm -f demo-mongo

# Step 6: Create a NEW container with the SAME volume
docker run -d --name demo-mongo2 -v demo-vol:/data/db mongo:6.0

# Step 7: Check if data survived
docker exec -it demo-mongo2 mongosh --eval "
  use testdb;
  db.demos.find().pretty()
"
```

```
"
```

```
# Your data is still there!
```

5 Key Advantages of Volumes (Interview Critical)

1. **Persistence:** Data survives container restarts and redeployments — critical for databases
2. **Separation of Concerns:** The container (application) remains stateless; the volume holds the state. This is the foundation of the "12-factor app" philosophy
3. **Security:** Granular control over exactly what is being mounted into the container — databases don't need to be exposed externally
4. **Performance:** Volumes outperform bind mounts, especially on Windows/Mac. *Why?* Docker Volumes are managed entirely by Docker, stored in a dedicated part of the host filesystem. The Docker engine has direct, optimized access. Bind Mounts must bridge across the host filesystem or a VM layer (Docker Desktop), adding I/O latency
5. **Backup/Restore:** Because data lives outside the container, snapshots and backups can be taken without touching the container itself

2.4 — Docker Networking: Deep Dive

The Core Problem: Container Isolation

Every Docker container runs in its own isolated **network namespace** — it has its own network stack, IP address, and routing table. This is great for security, but it means containers cannot talk to each other by default.

The #1 Beginner Mistake — The Localhost Misconception:

```
Outside a container:  localhost = your host machine
```

```
Inside a container:   localhost = THAT SPECIFIC CONTAINER ONLY
```

```
Result: From inside the frontend container, calling localhost:5000  
to reach the backend FAILS - the backend process doesn't exist  
inside the frontend container's isolated environment.
```

Docker Network Types

1. Default Bridge Network (Not Recommended for Production): When you start a container without specifying a network, Docker assigns it to the default bridge. Containers here can communicate but must use IP addresses — no DNS resolution by container name.

2. User-Defined Bridge Network (Recommended):

```
docker network create mern-net
```

- Has an embedded **DNS server** — containers can call each other by **container name** instead of IP
- `frontend` can call `http://backend:5000` instead of `http://172.17.0.3:5000`
- IPs change when containers restart; container names are stable

3. Host Network:

- Container shares the host's network directly — no isolation
- Linux only; used for performance-sensitive scenarios

4. None Network:

- No networking at all — used for security-sensitive, network-isolated containers

How Container DNS Works (The "Magic")

When you create a user-defined network called `mern-net` and connect containers to it:

Container "mongo" → DNS name: "mongo" → resolves to its current IP

Container "backend" → DNS name: "backend" → resolves to its current IP

Container "frontend" → DNS name: "frontend" → resolves to its current IP

So in `docker-compose.yml`:

`MONGO_URI=mongodb://mongo:27017/dev_db` ← "mongo" is the

DNS name

VITE_API_URL=http://backend:5000/api

← "backend" is the

DNS name

Docker runs an internal DNS resolver on `127.0.0.11` within each container that resolves container names to their current IP addresses automatically.

Networking Commands

```
# List all networks
```

```
docker network ls
```

```
# Create a custom network
```

```
docker network create mern-net
```

```
# Run containers on the same network
```

```
docker run -d --name mongo --network mern-net mongo:6.0
```

```
docker run -d --name backend --network mern-net -e
```

```
MONGO_URI=mongodb://mongo:27017/db my-backend
```

```
# From inside the backend container, mongo:27017 is reachable!
```

```
# Inspect network details (see all connected containers and their  
IPs)
```

```
docker network inspect mern-net
```

```
# Remove a network (all containers must be disconnected first)
```

```
docker network rm mern-net
```

Why Container Networking Matters for DevOps (Interview)

Q: "Why can't containers talk to each other by default?" A: Each container operates within its own isolated network namespace for security and resource management. To enable inter-container communication, they must be attached to the same shared user-defined network.

4 Advantages of Container Networking:

1. **Service Discovery:** IP addresses change on container restarts, but container names (DNS names) remain constant
2. **Security:** Databases can be hidden inside the `mern-net` network, invisible to the outside world, while only the backend can connect to them

3. **Port Collision Prevention:** Two different containers can both use port 5000 internally without conflict, because they're in isolated namespaces
 4. **Scalability:** Multiple backend containers can sit behind the same backend DNS name, enabling load distribution
-

2.5 — Docker Registry

Why a Registry is Necessary

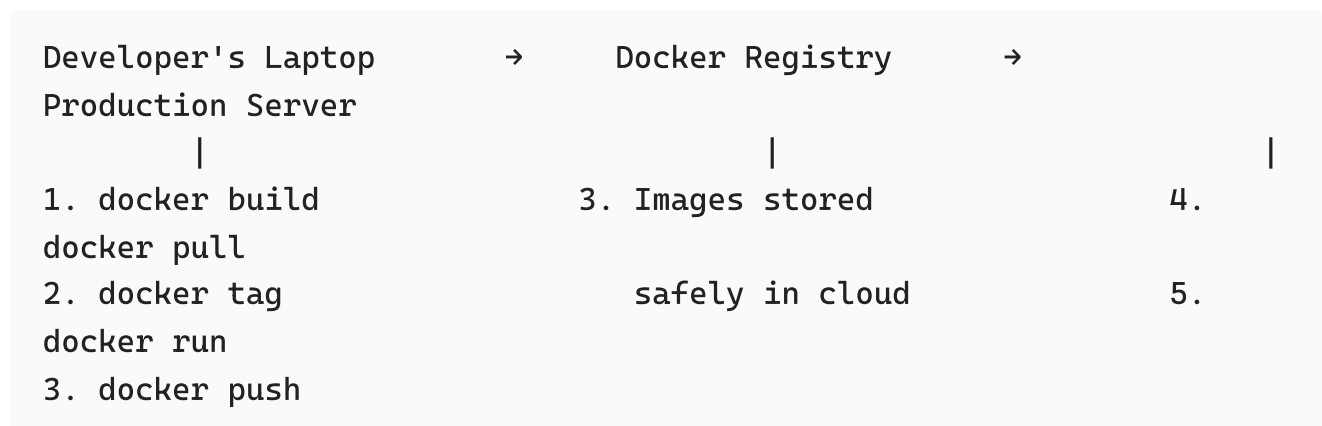
When you build a Docker image with `docker build`, it lives only on your local machine. To share it with teammates, deploy it on an EC2 server, or have a Kubernetes cluster pull it, you need a central place to store it.

A Docker Registry is GitHub for Docker images.

Popular Registries:

- **Docker Hub** — Public registry (free for public repos, like GitHub public repos)
- **GitHub Container Registry (GHCR)** — Integrated with GitHub repositories
- **Amazon ECR (Elastic Container Registry)** — AWS's private registry
- **Google Container Registry** — Google Cloud's registry

The Registry Workflow



Hands-On Registry Demo

```
# Step 1: Build and tag for Docker Hub
docker build -t mern-backend:local ./server
docker tag mern-backend:local yourusername/mern-backend:1.0.0

# Step 2: Login and push
```

```
docker login
docker push yourusername/mern-backend:1.0.0

# Step 3: On any other machine, pull and run
docker pull yourusername/mern-backend:1.0.0
docker run -p 5000:5000 yourusername/mern-backend:1.0.0
```

Image Tagging Strategy:

```
1.0.0          ← Semantic versioning for stable releases
latest         ← Points to the most recent stable build
v1.0.0-build.123 ← Include CI build number for traceability
```

Security Considerations:

- Use **private registries** for proprietary application code
- Scan images for vulnerabilities with tools like Trivy or Docker Scout
- Use **minimal base images** (Alpine Linux reduces attack surface and image size)

2.6 — MERN Full-Stack Docker Implementation

The Complete docker-compose.yml for MERN + Volumes + Networking

```
services:
  mongo:
    image: mongo:6.0
    container_name: mongo
    restart: unless-stopped
    volumes:
      - mongo-data:/data/db          # Persistent storage for MongoDB
    networks:
      - mern-net

  backend:
    build:
      context: ./server
      dockerfile: Dockerfile
    container_name: backend
```



```

restart: unless-stopped
env_file:
  - ./server/.env
ports:
  - "5000:5000"
depends_on:
  - mongo
networks:
  - mern-net

frontend:
  build:
    context: ./client
    dockerfile: Dockerfile
    args:
      VITE_API_URL: http://localhost:5000/api    # Baked into
static build
  container_name: frontend
  restart: unless-stopped
  ports:
    - "5173:5173"
  depends_on:
    - backend
  networks:
    - mern-net

volumes:
  mongo-data:          # Named volume survives docker compose down

networks:
  mern-net:
    driver: bridge      # User-defined bridge with DNS resolution

```

How the Containers Communicate

Backend connects to Mongo:

```
MONGO_URI = mongodb://mongo:27017/dev_db
```

("mongo" is the container name → Docker DNS resolves it)

Frontend connects to Backend:

```
VITE_API_URL = http://localhost:5000/api
```

(From the host browser; for container-to-container, use "backend:5000")

To test API persistence:

```
curl -X POST http://localhost:5000/api/tasks \  
  -H "Content-Type: application/json" \  
  -d '{"title": "Test", "completed": false}'
```

```
# Restart backend container  
docker compose stop backend  
docker compose rm -f backend  
docker compose up -d backend
```

```
# Data still exists because Mongo is in a volume!  
curl http://localhost:5000/api/tasks
```

PHASE 3: AUTOMATION & THE CLOUD

CI/CD Logic, GitHub Actions, Jenkins, and EC2

3.1 — CI/CD Pipelines: Deep Dive

What & Why

What Continuous Integration (CI) is: A development practice where code changes are automatically integrated, built, linted, and tested every time a developer pushes to a shared repository.

What Continuous Deployment/Delivery (CD) is: The practice of automatically releasing code changes to production (or a staging environment) after passing all CI checks.

The Old Way (The Problem):

- Developers pushed directly to `main`, breaking production for all users
- Integration conflicts: 5 developers worked independently for weeks, then spent days resolving merge conflicts
- Manual deployments: SSH into a server, `git pull`, `npm install`, restart — tedious and error-prone

- No automated quality gates: broken code could reach production

What CI/CD Solves:

Old: Code → Manual PR review → Merge → Someone manually deploys → Hope it works

New: Code → Automated CI checks → Auto-merge if passing → Auto-deploy to server → Verified

The CI Workflow Logic

Developer pushes to feature branch

↓

GitHub Actions detects the push (Workflow Trigger)

↓

Runner: ubuntu-latest spins up

↓

Steps execute:

1. Checkout repo
2. Install Node.js
3. npm ci (clean install)
4. npm run lint -- --max-warnings=0
5. docker build (verify image builds)

↓

↓ PASS

↓ FAIL

PR marked safe to merge

Developer alerted, PR blocked

↓

Code merges to main

↓

CD job triggers (only if CI passed)

↓

SSH into EC2

↓

git pull + docker compose up --build

↓

Live app updated

The Three-Rule CI Safety Net

1. **Branching Strategy:** Developers never push directly to `main`. All work happens on feature branches.
 2. **Pull Requests (PRs):** Code can only reach `main` via a PR, which triggers automated checks.
 3. **Branch Protection Rules:** GitHub's "Require status checks to pass before merging" setting blocks the merge button if CI fails.
-

3.2 — GitHub Actions: CI Implementation

Setup

```
Repository Root
├── .github/
│   └── workflows/
│       └── ci.yml    ← Your automation script
```

ESLint & Prettier Setup (Code Quality Gates)

ESLint catches syntax errors, unused variables, and enforces coding rules.

Prettier enforces consistent formatting (semi-colons, quotes, print width).

Together, they ensure every commit that reaches `main` meets your team's quality standards.

Backend ESLint Config (`server/eslint.config.js`):

```
export default [
  {
    files: ['**/*.js'],
    rules: {
      semi: 'error',           // Require semicolons
      'no-unused-vars': 'warn', // Warn on unused variables
    },
  },
];
```

Frontend ESLint Config (`client/eslint.config.js`):

```
import js from '@eslint/js';
import globals from 'globals';
```

```

import reactHooks from 'eslint-plugin-react-hooks';
import reactRefresh from 'eslint-plugin-react-refresh';

export default [
  { ignores: ['dist'] },
  {
    files: ['**/*.js,jsx'],
    languageOptions: {
      ecmaVersion: 'latest',
      sourceType: 'module',
      globals: globals.browser,
      parserOptions: { ecmaFeatures: { jsx: true } },
    },
    plugins: {
      'react-hooks': reactHooks,
      'react-refresh': reactRefresh,
    },
    rules: {
      ...js.configs.recommended.rules,
      ...reactHooks.configs.recommended.rules,
      'no-unused-vars': ['warn', { varsIgnorePattern: '^[A-Z_]' }],
      'react-refresh/only-export-components': ['warn', {
allowConstantExport: true }],
    },
  },
];

```

Prettier Config (.prettierrc — same for both):

```

{ "semi": true, "singleQuote": true, "printWidth": 100 }

```

Package Scripts (add to both client/package.json and server/package.json):

```

"scripts": {
  "lint": "eslint . --ext .js,.jsx",
  "lint:fix": "eslint . --ext .js,.jsx --fix",
  "format": "prettier --write ."
}

```

The CI Workflow File (`.github/workflows/ci.yml`)

```
name: build-and-lint

on:
  push:
    branches: [main, dev]          # Trigger on push to these
    branches
  pull_request:
    branches: [main, dev]          # Trigger on PRs targeting these
    branches

jobs:
  build-and-lint:
    runs-on: ubuntu-latest          # Fresh Linux VM for each run

    steps:
      - name: Checkout repo
        uses: actions/checkout@v3  # Pull the code into the runner

      - name: Setup Node
        uses: actions/setup-node@v3
        with:
          node-version: 20

      - name: Install backend deps
        working-directory: server
        run: npm ci                  # Clean install (uses package-
        lock.json exactly)

      - name: Lint backend
        working-directory: server
        run: npm run lint -- --max-warnings=0  # Fail if ANY
        warnings exist

      - name: Install frontend deps
        working-directory: client
        run: npm ci

      - name: Lint frontend
        working-directory: client
        run: npm run lint -- --max-warnings=0
```

```
- name: Build backend Docker image
  run: docker build -t backend ./server # Validate
Dockerfile works

- name: Build frontend Docker image
  run: docker build -t frontend ./client
```

`npm ci` vs `npm install`: `npm ci` (clean install) uses the `package-lock.json` exactly as-is, ensuring every CI run installs the exact same dependency versions. `npm install` might update `package-lock.json`, causing non-deterministic builds.

Setting Up Branch Protection

Navigate to: Repository Settings → Branches → Add Rule for `main`

Enable:

- ✓ **Require status checks to pass before merging** → Select your `build-and-lint` check
- ✓ **Require branches to be up to date before merging**

Effect: The GitHub UI will show a red ✗ and block the Merge button if the CI workflow fails. This prevents broken code from reaching production regardless of who tries to merge.

3.3 — Continuous Deployment (CD) with GitHub Actions

The Goal

Remove the manual process of SSHing into the EC2 server, pulling code, and restarting containers every time a developer merges a change.

GitHub Secrets Configuration

Because the deployment runs automatically (no human present), the GitHub runner needs secure, non-interactive access to your server. **Never hardcode these values in the YAML file.**

Navigate to: Repository → Settings → Secrets and variables → Actions → New secret

Secret Name	Value
EC2_HOST	Public IP of your AWS EC2 instance (e.g., 3.91.x.x)
EC2_USER	Linux username (e.g., ec2-user for Amazon Linux)
EC2_KEY	The full contents of your .pem private key file
EC2_APP_DIR	Path to your app on the server (e.g., /home/ec2-user/mern-app)

The CD Workflow File (.github/workflows/ci-cd.yml)

```
name: ci-cd

on:
  push:
    branches: [main]           # Only deploy when code merges to
                                main

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      # Step 1: Add the EC2 server's SSH fingerprint to
      known_hosts.
      # This bypasses the "Are you sure you want to connect?"
      interactive prompt
      # that would otherwise block the automated script.
      - name: Add EC2 host to known_hosts
        run: |
          mkdir -p ~/.ssh
          ssh-keyscan -H "$EC2_HOST" >> ~/.ssh/known_hosts
        env:
          EC2_HOST: ${ secrets.EC2_HOST }

      # Step 2: Write the private SSH key to a file on the runner.
      # chmod 600 is critical - SSH refuses to use keys with open
      permissions.
```



```

- name: Write SSH key
  run: |
    echo "${EC2_KEY}" > ec2_key.pem
    chmod 600 ec2_key.pem
  env:
    EC2_KEY: ${ secrets.EC2_KEY }

# Step 3: SSH into the EC2 instance and run the deployment
commands.
# The heredoc (<<EOF) sends a block of commands to the remote
server.
- name: Deploy on EC2
  env:
    EC2_HOST: ${ secrets.EC2_HOST }
    EC2_USER: ${ secrets.EC2_USER }
    EC2_APP_DIR: ${ secrets.EC2_APP_DIR }
  run: |
    ssh -i ec2_key.pem ${EC2_USER}@${EC2_HOST} << EOF
    set -e                                # Exit immediately
if any command fails
    cd "${EC2_APP_DIR}" || exit 1

    # Fetch and reset to the latest code
    git fetch --all
    git reset --hard origin/main          # Discard any local
changes

    # Rebuild and restart containers
    docker compose up -d --build

    # Remove dangling images (leftover layers from old
builds)
    docker image prune -f

    docker compose ps
EOF

```

The Deployment Flow (Step by Step):

1. Developer pushes code to `main` branch
2. GitHub Actions runner spins up on `ubuntu-latest`
3. Runner fetches the server's SSH fingerprint (handles the interactive prompt)
4. Runner writes the private `.pem` key to a file with correct permissions

5. Runner SSHes into EC2 and executes the remote deployment script
6. On EC2: `git reset --hard origin/main` syncs the latest code
7. On EC2: `docker compose up -d --build` rebuilds images and restarts containers
8. New version is live — no human intervention required

Verification:

```
# On EC2 to verify manually
docker compose ps

# Frontend accessible at:
http://<EC2_PUBLIC_IP>/

# Backend API at:
http://<EC2_PUBLIC_IP>:5000/api
```

3.4 — AWS EC2 Deployment

What EC2 Is

Elastic Compute Cloud (EC2) is AWS's core service for renting virtual servers (called **instances**) in the cloud. "Elastic" means you can scale resources up or down on demand.

Core Components:

Component	What It Is
AMI (Amazon Machine Image)	Pre-configured OS template (e.g., Ubuntu 22.04 LTS). Think of it as an OS "snapshot"
Instance Type	Hardware configuration (CPU, RAM). <code>t2.micro</code> is free-tier eligible
Security Groups	Virtual firewall rules defining which ports accept traffic
Key Pairs	SSH authentication — a <code>.pem</code> file grants access to your instance
Public IPv4	The internet-facing IP address of your instance

Launching an Instance (Step by Step)

1. Login → EC2 Dashboard → **Launch Instance**
2. **Name:** mern-app
3. **AMI:** Ubuntu 22.04 LTS (or Amazon Linux 2023)
4. **Instance type:** t2.micro (free tier)
5. **Key pair:** Create new → Download .pem file (store safely — you cannot re-download it)
6. **Security group rules:**
 - Port 22 (SSH) → Your IP only
 - Port 80 (HTTP) → Anywhere
 - Port 5000 (API) → Anywhere (for testing)
7. Launch → Note the **Public IPv4 address**

Connect via SSH from Windows

```
ssh -i "C:\Users\yourname\.ssh\keypair.pem" ec2-  
user@<EC2_PUBLIC_IP>  
  
# Fix Windows permission issue if SSH complains about key  
permissions:  
icacls "C:\Users\yourname\.ssh\keypair.pem" /inheritance:r  
icacls "C:\Users\yourname\.ssh\keypair.pem" /grant:r  
"$($env:USERNAME):(R)"
```

Installing Dependencies on EC2

```
# Update package list  
sudo yum update -y          # Amazon Linux  
# sudo apt update -y        # Ubuntu  
  
# Install Docker  
sudo yum install docker -y  
sudo service docker start  
sudo usermod -aG docker ec2-user  # Allow ec2-user to run Docker  
without sudo  
# Log out and back in for group change to take effect  
  
# Install Docker Compose (as a CLI plugin)  
sudo mkdir -p /usr/local/lib/docker/cli-plugins  
sudo curl -SL  
https://github.com/docker/compose/releases/download/v2.29.2/docker-
```

```
compose-linux-x86_64 \
  -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
docker compose version      # Verify

# Install Git
sudo dnf install -y git      # Amazon Linux 2023
# sudo apt install -y git    # Ubuntu
```

Deploying the MERN App

```
# 1. Clone the repository
git clone https://github.com/user/mern-app.git
cd mern-app

# 2. Create environment files (NEVER committed to Git)
nano server/.env
# Add:
PORT=5000
NODE_ENV=production
MONGODB_URI=<mongodb-atlas-connection-string>

nano client/.env
# Add:
VITE_API_URL=http://<EC2_PUBLIC_IP>:5000/api

# 3. Critical: Bind backend to 0.0.0.0 (not localhost)
#   In your Node.js app:
app.listen(PORT, '0.0.0.0', () => { ... })
#   This ensures the app is accessible from outside the container

# 4. Build and run
docker compose build
docker compose up -d
docker compose ps

# Verify logs
docker compose logs -f backend
```

Why 0.0.0.0 ? Inside a container, if your app binds to `localhost` or `127.0.0.1`, it only accepts connections from within that container itself. Binding to `0.0.0.0` means "accept connections on all network interfaces,"

which includes the container's virtual network interface — allowing external traffic to reach it through the EC2 public IP.

Verification:

```
Frontend: http://<EC2_PUBLIC_IP>/  
API:      http://<EC2_PUBLIC_IP>:5000/api/health
```

If API calls fail, check the client `.env` and rebuild:

```
docker compose up -d --build frontend
```

3.5 — Jenkins: CI/CD Automation Server

What Jenkins Is and What Problem It Solves

Jenkins is an open-source automation server that eliminates manual, error-prone deployment steps. Instead of manually cloning repos, running `npm install`, building Docker images, and deploying — Jenkins automates the entire flow.

Think of Jenkins as your personal DevOps robot that executes tasks exactly how you instruct it to, every time.

The Time Comparison:

Step	Without Jenkins	With Jenkins
Pull code	1 min (manual)	0 min (automated)
Install deps	3 min (manual)	0 min (automated)
Build Docker images	5 min (manual)	0 min (automated)
Push to registry	2 min (manual)	0 min (automated)
Deploy and test	5 min (manual)	0 min (automated)
Total developer time	16 min per deploy	~2 min per push

Key Jenkins Concepts

Concept	What It Is
Job (Project)	The basic unit of work. Freestyle = simple tasks. Pipeline = complex multi-step workflows
Build	One execution of a job. Each run is numbered (#1, #2, ...) for history tracking
Pipeline	A series of defined stages stored in a <code>Jenkinsfile</code> — automation as code
Node/Agent	The machine where the job executes. Default = Jenkins Master node. Can scale with external agents
Plugin	Extensions for Jenkins (GitHub integration, Docker support, Slack notifications)
Workspace	Temporary directory where Jenkins pulls source code and performs build operations
Executor	A worker slot that determines how many jobs can run concurrently on a node
Trigger	What causes a job to run: manual click, GitHub push (Webhook), or scheduled time

Jenkins Flow Architecture

1. Code Commit: Developer pushes to GitHub
↓
2. Detection: Jenkins detects via Webhook (recommended) or Polling (every 5 min)
↓
3. Checkout: Jenkins pulls source code into its workspace
↓
4. Execution: Runs Jenkinsfile stages (or shell commands for Freestyle jobs)
↓
5. Reporting: Saves logs/artifacts, sends email/Slack notifications

Webhook vs. Polling:

- **Webhook:** GitHub notifies Jenkins *immediately* when code is pushed — instant response, production-grade

- **Polling:** Jenkins checks GitHub every N minutes — adds latency, wastes resources, used when webhooks aren't available
-

Installing Jenkins with Docker (The Clean Way)

```
# Step 1: Run Jenkins container with Docker socket mount
# The Docker socket (/var/run/docker.sock) gives Jenkins access to
the host's Docker daemon
# This is how Jenkins can run docker build / docker run commands
from inside the container
docker run -d \
  --name jenkins \
  -p 8080:8080 \
  -p 50000:50000 \
  -v /var/run/docker.sock:/var/run/docker.sock \    # Critical:
Docker socket mount
  -v jenkins_home:/var/jenkins_home \              # Persistent
Jenkins data
  jenkins/jenkins:lts

# Step 2: Install Docker CLI inside the Jenkins container
docker exec -u 0 -it jenkins bash
apt-get update && apt-get install -y docker.io
exit

# Step 3: Restart Jenkins to apply changes
docker restart jenkins

# Step 4: Fix Docker socket permissions (if permission errors
occur)
docker exec -u 0 jenkins chmod 666 /var/run/docker.sock

# Step 5: Verify Docker is available inside Jenkins
docker exec -it jenkins docker --version
docker exec -it jenkins docker ps -a

# Step 6: Get the initial admin password
docker exec jenkins cat
/var/jenkins_home/secrets/initialAdminPassword
```

The Architecture (Why the Docker Socket Mount Works):

```
[ Your Laptop (Host Machine) ]
    |
    [ Docker Engine ]
    ↑
    | (via /var/run/docker.sock = a Unix socket, a
communication pipe)
    |
    [ Jenkins Container ]
    - Jenkins app
    - Docker CLI (just the commands, not the engine)
```

When Jenkins runs "docker build":

- The command goes through `docker.sock`
- Reaches the HOST's Docker daemon
- The real Docker engine builds the image
- Result is available on the host

Jenkins Admin Setup

1. Open `http://localhost:8080`
2. Paste the initial admin password
3. Choose **"Install suggested plugins"**
4. Create a First Admin User

Jenkins UI Key Areas

- **New Item:** Entry point for creating Freestyle projects or Pipelines
- **Build History:** Track status of past builds (Green = success, Red = failure, Yellow = unstable)
- **Manage Jenkins** → **Plugins:** Install GitHub, Docker, Slack integrations
- **Manage Jenkins** → **Nodes:** Configure build agents
- **Manage Jenkins** → **Credentials:** Securely store API keys, passwords, SSH keys (never hardcode in Jenkinsfiles)

Freestyle Jobs vs. Pipeline Jobs

	Freestyle Job	Pipeline Job
Configuration	GUI-based	Code (Jenkinsfile)
Version control	Not version-controlled	Stored in Git with your app
Complexity	Good for simple tasks	Handles complex multi-stage workflows
Reproducibility	Harder to replicate	Fully reproducible from code
Visibility	Limited	Full stage view in Jenkins UI

Job Triggers

```
// Manual: Click "Build Now" – good for testing

// SCM Polling: Jenkins checks GitHub every 5 minutes
H/5 * * * * *

// Scheduled: Every night at 2AM
H 2 * * * *

// GitHub Webhook (Production Standard):
// GitHub → Settings → Webhooks → http://jenkins:8080/github-
webhook/
// Triggers immediately on push
```

Complete MERN Jenkins Pipeline (Jenkinsfile)

`docker-compose.yml` (used in Jenkins pipeline):

```
services:
  mongo:
    image: mongo:6.0
    container_name: mongo
    restart: unless-stopped
    volumes:
      - mongo-data:/data/db
    networks:
```

```

    - mern-net

backend:
  build:
    context: ./server
    dockerfile: Dockerfile
  container_name: backend
  restart: unless-stopped
  env_file:
    - ./server/.env
  ports:
    - "5000:5000"
  depends_on:
    - mongo
  networks:
    - mern-net

frontend:
  build:
    context: ./client
    dockerfile: Dockerfile
  args:
    VITE_API_URL: http://localhost:5000/api
  container_name: frontend
  restart: unless-stopped
  ports:
    - "5173:5173"
  depends_on:
    - backend
  networks:
    - mern-net

volumes:
  mongo-data:

networks:
  mern-net:
    driver: bridge

```

Jenkinsfile (repository root):

```

pipeline {
  agent any

```

```

// Environment variables available to all stages
environment {
    FRONTEND_IMAGE = "mern-frontend:jenkins"
    BACKEND_IMAGE  = "mern-backend:jenkins"
    PORT           = "5000"
    MONGO_URI      = "mongodb://mongo:27017/taskdb"
    DOCKER_REGISTRY = "your-dockerhub-username"
}

stages {

    stage('Checkout Code') {
        steps {
            // Jenkins automatically clones the repo containing this
            Jenkinsfile
            git url: 'https://github.com/user/mern-app.git', branch:
            'main'
            script {
                echo "Building commit: ${env.GIT_COMMIT}"
                echo "Branch: ${env.GIT_BRANCH}"
            }
        }
    }

    stage('Prepare .env') {
        steps {
            sh '''
                mkdir -p server
                cat > server/.env <<EOF
                PORT=$PORT
                MONGO_URI=$MONGO_URI
                EOF
            '''
        }
    }

    stage('Build Docker Images') {
        steps {
            sh '''
                echo "Building backend image..."
                docker build -t $BACKEND_IMAGE ./server

                echo "Building frontend image..."
            '''
        }
    }
}

```

```

        docker build -t $FRONTEND_IMAGE ./client \
            --build-arg VITE_API_URL=http://localhost:5000/api
    ...
}
}

stage('Run with Docker Compose') {
    steps {
        sh '''
            echo "Starting MERN stack with Docker Compose..."
            docker compose up -d

            echo "Showing running containers..."
            docker ps

            echo "==== Backend Logs ====="
            docker logs backend || true

            echo "==== Frontend Logs ====="
            docker logs frontend || true
        ...
    }
}

stage('Health Check & Testing') {
    steps {
        sh '''
            # Wait for services
            sleep 30

            # Test backend
            curl -f http://localhost:5000/health || {
                echo "Backend health check failed"
                exit 1
            }

            # Test frontend
            curl -f http://localhost:5173 || {
                echo "Frontend health check failed"
                exit 1
            }

            # Test DB connection
            curl -f http://localhost:5000/api/tasks || {

```

```

        echo "Database connection failed"
        exit 1
    }
    ...
}
}

stage('Integration Tests') {
    steps {
        sh '''
            # Create a test task
            curl -X POST http://localhost:5000/api/tasks \
                -H "Content-Type: application/json" \
                -d '{"title": "Jenkins CI Test Task", "completed":
false' \
                -f || exit 1

            # Verify task was created
            curl http://localhost:5000/api/tasks | grep "Jenkins CI
Test Task" || exit 1
            ...
        '''
    }
}

stage('Push to Registry') {
    when {
        branch 'main' // Only push images when building from
main
    }
    steps {
        script {
            withCredentials([usernamePassword(credentialsId:
'dockerhub-creds',
                                usernameVariable:
'USERNAME',
                                passwordVariable:
'PASSWORD')]) {
                sh 'docker login -u $USERNAME -p $PASSWORD'
                sh "docker tag ${BACKEND_IMAGE}
${DOCKER_REGISTRY}/mern-backend:${BUILD_NUMBER}"
                sh "docker push ${DOCKER_REGISTRY}/mern-
backend:${BUILD_NUMBER}"
                sh "docker tag ${FRONTEND_IMAGE}
${DOCKER_REGISTRY}/mern-frontend:${BUILD_NUMBER}"
            }
        }
    }
}

```

```

        sh "docker push ${DOCKER_REGISTRY}/mern-frontend:${BUILD_NUMBER}"
    }
}
}
}
}

// Post-build actions (always run regardless of success/failure)
post {
    always {
        sh '''
            docker compose down || true
            docker image prune -f || true
        '''
    }
    success {
        echo "Pipeline #${BUILD_NUMBER} succeeded!"
    }
    failure {
        sh 'docker compose logs || true'
        echo "Pipeline failed! Check logs above."
    }
}
}
}

```

Setting Up the Pipeline Job in Jenkins UI

1. **New Item** → Name: `mern-ci-cd-pipeline` → **Pipeline** → OK
 2. **General**: Add description
 3. **Build Triggers**: Check "GitHub hook trigger for GITScm polling"
 4. **Pipeline**:
 - Definition: **Pipeline script from SCM**
 - SCM: **Git**
 - Repository URL: `https://github.com/user/mern-app.git`
 - Branch: `*/main`
 - Script Path: `Jenkinsfile`
 5. Save → **Build Now** for first manual run
-
-

PHASE 4: SCALING & ORCHESTRATION

Nginx Reverse Proxy and Kubernetes

4.1 — Nginx Reverse Proxy: Deep Dive

What & Why

What Nginx is: A high-performance web server and **reverse proxy server** built on an event-driven, asynchronous architecture. It can handle thousands of concurrent connections with minimal resource usage — this is why 30%+ of websites use it.

The Old Way (The Problem): In our basic Docker Compose setup:

- Frontend exposed on port 5173
- Backend API exposed on port 5000
- Users need to know BOTH ports: `http://server:5173` for UI, `http://server:5000/api` for API
- No SSL termination, no caching, no rate limiting, no load balancing
- Every microservice requires its own port — unmanageable at scale

What Nginx Reverse Proxy Solves:

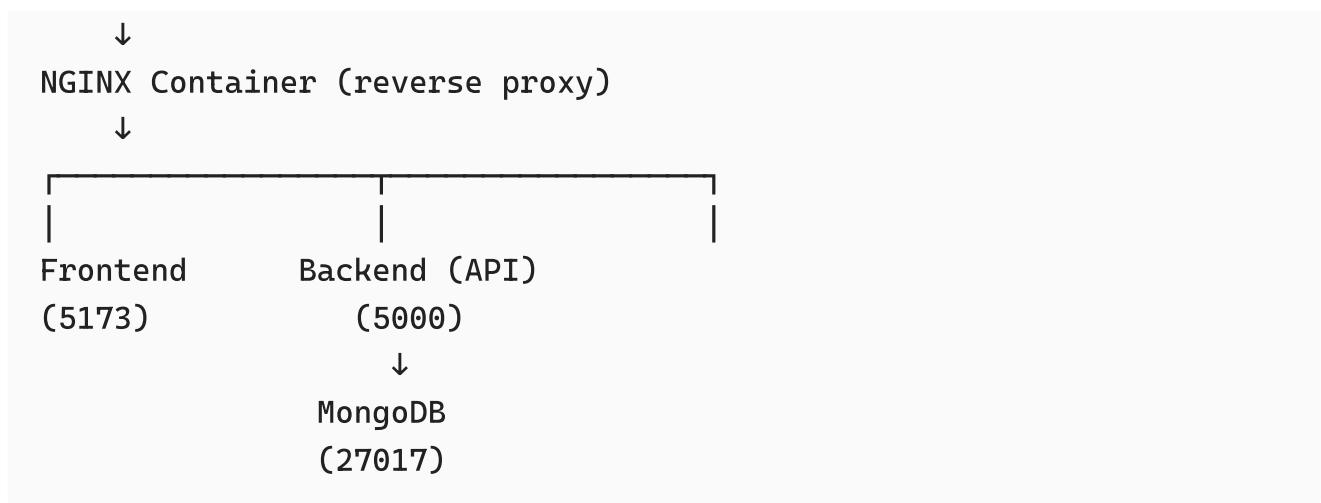
- **Single entry point:** Everything behind port 80 (standard HTTP)
- **Path-based routing:** `/` → frontend, `/api` → backend
- **Hides backend complexity:** Clients don't know how many servers exist
- **Security:** Internal services never directly exposed to the internet
- **Performance:** Caching, gzip compression, serving static files efficiently
- **Foundation for Kubernetes Ingress:** Nginx Ingress Controller (used in K8s) works on the exact same principles

The "Where" in MERN Deployment

User Browser



`http://localhost:80` (single clean URL)



The Architecture

Reverse Proxy vs. Forward Proxy:

Forward Proxy	Reverse Proxy (Nginx)	
Works on behalf of	The client	The server
Flow	Client → Proxy → Internet	Client → Proxy → Private Services
Client sees	Proxy's IP (client is hidden)	Proxy (servers are hidden)
Use cases	VPNs, corporate networks, content filtering	Web servers, load balancing, SSL termination

Real-world analogies:

- Netflix uses Envoy (similar to Nginx) for service-to-service communication
- GitHub uses Nginx to serve static files at the edge
- Every cloud-native deployment uses a reverse proxy in front of microservices

nginx.conf (The Routing Brain)

```
# events block is required (even if empty)
events {}

http {
```



```

# Allow larger request bodies (e.g., file uploads up to 10MB)
client_max_body_size 10M;

server {
    listen 80;      # Nginx listens on standard HTTP port

    # Route frontend requests
    # All requests to "/" go to the React/Vite frontend
    location / {
        proxy_pass http://frontend:5173;          # Docker DNS
        resolves "frontend"
        proxy_set_header Host $host;              # Pass original
        Host header
        proxy_set_header X-Real-IP $remote_addr;   # Pass client's
        real IP
        proxy_set_header X-Forwarded-Proto $scheme; # Pass protocol
        (http/https)
        proxy_buffering off;                       # Disable
        buffering for SSE/WebSockets
    }

    # Route API requests to backend
    # All /api requests go to the Node.js backend
    location /api {
        proxy_pass http://backend:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 5s;    # Fail fast if backend
        unreachable
        proxy_send_timeout 60s;      # Allow long-running requests
        proxy_read_timeout 60s;
    }

    # Internal health check endpoint
    location = /nginx-health {
        return 200 "OK";
        add_header Content-Type text/plain;
    }
}

```

MERN + Nginx docker-compose.yml

The critical change is that **frontend and backend no longer expose their ports directly to the host** — only Nginx does:

```
services:
  mongo:
    image: mongo:6.0
    container_name: mongo
    restart: unless-stopped
    volumes:
      - mongo-data:/data/db
    networks:
      - mern-net

  backend:
    build:
      context: ./server
      dockerfile: Dockerfile
    container_name: backend
    restart: unless-stopped
    env_file:
      - ./server/.env
    # NO "ports:" – backend is only accessible via Nginx, not
    # directly from outside
    depends_on:
      - mongo
    networks:
      - mern-net

  frontend:
    build:
      context: ./client
      dockerfile: Dockerfile
      args:
        VITE_API_URL: "/api" # KEY CHANGE: relative path, not
        absolute URL
    container_name: frontend
    restart: unless-stopped
    depends_on:
      - backend
    networks:
      - mern-net
```

```
nginx:
  image: nginx:alpine
  container_name: nginx
  restart: unless-stopped
  volumes:
    - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro # Mount
config as read-only
  ports:
    - "80:80" # ONLY Nginx is exposed to the host
  depends_on:
    - frontend
    - backend
  networks:
    - mern-net

volumes:
  mongo-data:

networks:
  mern-net:
    driver: bridge
```

The Critical VITE_API_URL Change: ARG vs ENV (Deep Dive)

This is one of the most misunderstood concepts in MERN + Docker setups.

The Rule: Vite replaces `import.meta.env.VITE_*` variables **at build time**. The built static HTML/CSS/JS files have the values baked in. Changing the environment variable after the build does NOTHING to the compiled JavaScript.

WRONG mental model:

Container starts → reads `VITE_API_URL` from env → passes to JS

CORRECT mental model:

docker build runs → Vite reads `VITE_API_URL` → bakes value into `/dist/bundle.js`

Container starts → serves pre-baked `bundle.js` (env vars irrelevant now)

In the Dockerfile:

```
# ARG is available ONLY at build time (docker build --build-arg
VITE_API_URL=/api)
ARG VITE_API_URL=/api

# ENV makes it available as an environment variable during `npm run
build`
# Vite reads process.env.VITE_API_URL during the build
ENV VITE_API_URL=$VITE_API_URL

# npm run build reads VITE_API_URL and bakes it into the bundle
RUN npm run build
```

Why /api is the correct value with Nginx:

- The browser calls /api/tasks (relative URL, no host/port)
- The request goes to http://merndemo.local/api/tasks (same origin)
- Nginx receives it and forwards to http://backend:5000/api/tasks
- **No CORS issues** because the browser never makes a cross-origin request

Without Nginx, if you used http://backend:5000 , the browser would:

1. Try to resolve backend — fails because backend is a Docker DNS name, not a public hostname
2. Even if you used the server's IP, the browser would make a cross-origin request (different port) → CORS errors

How Everything Works: Step-by-Step

```
# Start everything
docker compose up --build
```

Step 1: Docker builds images and starts containers

- Builds frontend (with VITE_API_URL=/api baked in) and backend images
- Pulls mongo:6.0 and nginx:alpine from Docker Hub
- Creates mern-net bridge network
- Starts all 4 containers

Step 2: Networking within mern-net

- All containers can resolve each other by name:
 - `http://backend:5000` from within the network
 - `http://frontend:5173` from within the network
 - `mongodb://mongo:27017` from within the network

Step 3: User opens browser

- Browser navigates to `http://localhost`
- Request hits Nginx (port 80:80 is mapped)

Step 4: Nginx routing decision

- Path `/` → proxies to `http://frontend:5173` → serves React app
- Path `/api/tasks` → proxies to `http://backend:5000/api/tasks` → returns JSON

Step 5: Backend → MongoDB

- Backend internally calls `mongodb://mongo:27017`
 - Works because `mongo` resolves via Docker DNS within `mern-net`
-

Interview Edge: Nginx Questions

Q: "What's the difference between a forward proxy and a reverse proxy?" A:

A forward proxy acts on behalf of the **client** — it hides the client from servers (e.g., VPN, corporate proxy). A reverse proxy acts on behalf of the **server** — it hides backend services from clients. Nginx in our setup is a reverse proxy: the browser only knows about Nginx, not the individual frontend/backend containers.

Q: "Why use Nginx in front of Docker Compose services instead of just exposing ports?" A: Single entry point on port 80; path-based routing without requiring clients to know service ports; ability to add SSL termination, rate limiting, compression, and caching in one place; security (internal services not directly exposed to the internet).

Q: "Why does VITE_API_URL need to be a build-time ARG rather than a runtime ENV?" A: Vite is a build tool that produces static HTML/CSS/JS. During `npm run build`, it reads `VITE_*` environment variables and replaces

`import.meta.env.VITE_API_URL` with the actual string value in the compiled JavaScript. Once built, the value is hardcoded in the bundle. Runtime container environment variables have no effect on already-compiled JavaScript.

4.2 — Kubernetes Orchestration: Deep Dive

What & Why

What Kubernetes (K8s) is: An open-source container orchestration platform, originally built by Google, now maintained by the CNCF (Cloud Native Computing Foundation). It manages containers for you — deciding where they run, keeping them alive, scaling them up/down, and enabling communication between them.

| The name: K8s = K + 8 letters + s

The Old Way (What Docker Compose + EC2 Cannot Solve):

Problem	Without K8s
Backend crashes	Site goes down until you SSH in and restart manually
Traffic spike (1k → 50k users)	No auto-scaling — server dies under load
Frontend needs backend address	You hardcode IP/PORT — breaks when container restarts
10 microservices	Manually managing 10 containers across multiple machines

The Declarative Philosophy: Instead of imperative commands like "start 3 containers, if one dies start another, route traffic to all three...", you write a YAML file declaring "I want 3 replicas of this container always running." K8s reads that and **continuously works to make reality match your declaration** — forever.

The "Where" in MERN Deployment

Developer pushes code



CI/CD Pipeline (GitHub Actions / Jenkins)



Docker images pushed to Registry



Kubernetes Cluster (pulls images)

├─ Control Plane (Master Node)

│ ├─ API Server

│ ├─ etcd

│ └─ Controller Manager

│ └─ Scheduler

└─ Worker Nodes

├─ Pod: MongoDB (with PVC)

├─ Pod: Backend (×2 replicas)

└─ Pod: Frontend (×1 replica)



Ingress Controller (Nginx)



User's Browser

The Architecture: Two Halves of a K8s Cluster

Master Node (Control Plane) — "The Brain"

The control plane makes global decisions about the cluster. It **never runs your actual application** — it only manages everything.

API Server → "The front desk / reception"

- Single entry point for ALL requests
- Every kubectl command hits the API Server first
- Validates and processes requests, then updates etcd

etcd → "The memory / database"

- Distributed key-value store
- Stores the ENTIRE cluster state: every pod, service, deployment, configmap
- If the API Server is the front desk, etcd is the filing cabinet behind it
- Critical: if etcd data is lost, the cluster loses its mind

Controller Manager → "The watchdog"

- Continuously watches current state vs desired state

- If a pod crashes: "Desired = 3 replicas, Current = 2 - spin up a new one!"
- This is what gives K8s its self-healing property
- Contains multiple controllers: ReplicaSet controller, Deployment controller, etc.

Scheduler → "The dispatcher"

- Once the Controller Manager says "we need a new pod," the Scheduler decides WHERE
- Picks the best available worker node based on available CPU, RAM, and constraints
- Places the pod on that node

Worker Node — "Where Your App Actually Runs"

Kubelet → "The node's manager"

- Agent running on EVERY worker node
- Talks to the master's API Server
- Ensures containers in pods are running as instructed
- Reports node/pod health back to the master

kube-proxy → "The node's network cop"

- Handles node-level networking rules
- Manages iptables/IPVS rules for routing Service traffic to pods
- Provides basic load balancing

Pod → "The smallest deployable unit"

- Wraps one (or more) containers
- All containers in a pod share the same IP and volumes
- Pods are ephemeral: if a pod dies, K8s recreates a NEW one (with a new IP)
- Think of a pod as a single house - if it burns down, a new one gets built

Container → "What actually runs inside the pod"

- Your Docker container
- Multiple containers in the same pod can communicate via localhost

The Full Request Flow


```
kubectl apply -f deployment.yaml
```

↓

API Server → validates request, authenticates user

↓

etcd → stores the desired state (e.g., "3 replicas of backend")

↓

Controller Manager → detects gap: "desired=3, actual=0"

↓

Scheduler → picks best worker node (most available CPU/RAM)

↓

Worker Node (Kubelet) → pulls image, creates pod, starts container

↓

kube-proxy → updates networking rules so traffic can reach the pod

↓

App running! Kubelet reports "Ready" back to API Server

Core K8s Objects

Pod — "The Single House"

The smallest deployable unit. Rarely created directly — Deployments manage them.

```
# Never create pods manually (no self-healing). Use Deployments.
apiVersion: v1
kind: Pod
metadata:
  name: backend-pod
spec:
  containers:
    - name: backend
      image: mern-backend:local
      ports:
        - containerPort: 5000
```

ReplicaSet — "The Reliability Layer"

Ensures N identical pod replicas are always running. If a pod crashes, ReplicaSet recreates it. You rarely write ReplicaSets directly — Deployments manage them.

```

ReplicaSet backend-rs (replicas: 3)
├─ Pod backend-1 [Running]
├─ Pod backend-2 [Running] ← Pod 2 crashes
└─ Pod backend-3 [Running]
    ↓ Controller Manager detects: "2 running, 3 desired"
├─ Pod backend-1 [Running]
├─ Pod backend-2 [Recreated] ← New pod automatically started
└─ Pod backend-3 [Running]

```

Deployment — "The Management Layer"

The most common object you'll create. Manages ReplicaSets, handles rolling updates and rollbacks.

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: devops-part3
  labels:
    app: backend
spec:
  replicas: 2                                # Always keep 2 pods running
  selector:
    matchLabels:
      app: backend                          # Manage pods with this label
  template:
    metadata:
      labels:
        app: backend                       # Label applied to pods (must
match selector)
    spec:
      containers:
        - name: backend
          image: backend:local
          imagePullPolicy: IfNotPresent    # Use local image, don't
pull from registry
          ports:
            - containerPort: 5000
          env:
            - name: MONGO_URI
              value: "mongodb://mongo:27017/taskdb"
          readinessProbe:                  # Only route traffic when

```

```

container is truly ready
  httpGet:
    path: /health
    port: 5000
  initialDelaySeconds: 3
  periodSeconds: 5
  failureThreshold: 3
  livenessProbe:           # Restart container if it becomes
unresponsive
  httpGet:
    path: /health
    port: 5000
  initialDelaySeconds: 10
  periodSeconds: 10

```

Why readiness vs liveness probes?

- **Readiness Probe:** Controls *traffic routing* — pod won't receive traffic until this passes. Prevents sending requests to a container that hasn't connected to its database yet.
- **Liveness Probe:** Controls *pod restart* — if this fails, kubelet kills and restarts the container. Handles deadlocked processes that are "running" but not functional.

Service — "The Stable Address / Main Gate"

Since pods are ephemeral and get new IPs every time they're recreated, Services provide a **stable DNS name and virtual IP** that always routes to the correct (healthy) pods.

```

apiVersion: v1
kind: Service
metadata:
  name: backend           # This becomes the DNS name within
the cluster
  namespace: devops-part3
spec:
  selector:
    app: backend          # Routes to pods with this label
  ports:
    - protocol: TCP
      port: 5000           # Port clients use to reach the
Service

```

```
targetPort: 5000           # Port on the pod container
type: ClusterIP            # Internal-only (default)
```

How Services find Pods: Services don't use IP addresses — they use **label selectors**. The Service continuously watches for pods matching its selector labels and updates its endpoint list. When a pod dies and is recreated (even with a new IP), the Service automatically discovers the new pod via its labels.

Service Types:

Type	Accessibility	Use Case
ClusterIP (default)	Internal cluster only	Pod-to-Pod communication (backend → mongo)
NodePort	External via high port (30000-32767)	Testing/dev, not production- friendly
LoadBalancer	External via cloud load balancer	Production on AWS/GCP/Azure
Headless	Direct pod discovery, no virtual IP	StatefulSets, direct pod access

Imperative vs. Declarative (The Paradigm Shift)

Imperative (Basic — Avoid in Production):

```
kubectl run backend-pod --image=mern-backend:dev --port=5000
kubectl set env deployment/backend MONGODB_URI="<atlas-uri>"
kubectl scale deployment backend --replicas=2
```

Problems: Not version-controlled, hard to reproduce, error-prone at scale.

Declarative (Standard):

```
kubectl apply -f k8s/02-backend.yaml
```

Benefits:

- **Idempotent:** Re-applying the same YAML = same state, no surprises

- **Version-controlled:** YAML in Git = infrastructure-as-code — reviewable, reversible
 - **Automation/GitOps:** Tools like ArgoCD watch Git and auto-apply YAML changes
 - **Drift detection:** Controllers continuously reconcile actual vs. desired state
 - **Environment promotion:** Promote the same YAML from dev → staging → prod
-

Namespaces: Virtual Clusters

Namespaces are organizational "folders" within a Kubernetes cluster, used to isolate resources.

```
apiVersion: v1
kind: Namespace
metadata:
  name: devops-part3
```

Why use namespaces?

- Avoid naming conflicts (a `backend` deployment in `dev` vs `backend` in `prod`)
 - Apply resource quotas and access policies per namespace
 - Easy cleanup: `kubectl delete namespace devops-part3` removes ALL demo resources
-

Labels and Selectors: How K8s Objects Find Each Other

Labels are key-value pairs attached to K8s objects. Selectors are queries that use labels to find objects.

```
# On the Deployment's pod template:
labels:
  app: backend
  version: "1.0"

# On the Service:
selector:
```

```
app: backend    # Routes to ALL pods with this label

# This is how Service → Pod routing works:
# Service doesn't know pod IPs (they change). It continuously
queries:
# "Give me all pods with label app=backend" → those are its
endpoints.
```

Troubleshooting tip: If a Service has no endpoints (no traffic routing), 90% of the time it's a label mismatch — the selector in the Service doesn't match the labels on the pod template.

ConfigMaps and Secrets

Why Configurations Matter

Hardcoding config (like `MONGO_URI` in the Deployment YAML) is bad because:

- You need to rebuild images if config changes
- You risk committing secrets to Git
- You can't easily separate dev/staging/prod configurations

ConfigMap (Non-Sensitive Data)

```
# k8s/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: devops-part3
data:
  MONGO_HOST: "mongo"
  LOG_LEVEL: "debug"
  FRONTEND_BASE: "/"
```

Not encrypted — visible to anyone with cluster access. Use for URLs, feature flags, hostnames.

Secret (Sensitive Data)

```
# k8s/secret-mongo.yaml
apiVersion: v1
kind: Secret
metadata:
  name: mongo-secret
  namespace: devops-part3
type: Opaque          # Generic key-value secret
data:
  # Values MUST be base64-encoded:
  # echo -n "mongodb://mongo:27017/taskdb" | base64
  MONGO_URI: bW9uZ29kYjovL21vbmdvOjI3MDE3L3Rhc2tkYg==
```

Base64 is NOT encryption. It's just encoding. Anyone who can access the secret can decode it. For true security, integrate with HashiCorp Vault, AWS Secrets Manager, or Kubernetes external secrets operators. Never push secret YAML files with real values to public repos.

Injecting Config into a Deployment

```
spec:
  containers:
    - name: backend
      envFrom:
        - configMapRef:
            name: app-config          # Injects all configmap keys as
env vars
        - secretRef:
            name: mongo-secret       # Injects all secret keys as env
vars
```

Or inject specific keys:

```
env:
  - name: MONGO_URI
    valueFrom:
      secretKeyRef:
        name: mongo-secret
        key: MONGO_URI
```

Applying and verifying:

```
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/secret-mongo.yaml

# Verify they exist
kubectl -n devops-part3 get configmap app-config -o yaml
kubectl -n devops-part3 get secret mongo-secret -o yaml # Values
shown as base64

# Force pods to reload new config
kubectl -n devops-part3 rollout restart deployment/backend
```

Ingress: One Domain, Many Services

What Ingress Is and Why We Need It

ClusterIP — internal only, can't reach from browser. **NodePort** — works, but exposes ugly ports (30000-32767) per service. For a 10-service app: 10 different ports. Unacceptable for production. **Port-forwarding** — dev/debugging only, not scalable.

The NodePort Problem in Plain English:

```
Without Ingress:
  http://192.168.99.101:31000/    ← Frontend
  http://192.168.99.101:32000/api ← Backend

With Ingress:
  http://merndemo.local/          ← Frontend
  http://merndemo.local/api       ← Backend (same domain, path-based
routing)
```

This is how Amazon works: `amazon.com/cart` → cart service,
`amazon.com/payments` → payment service. One domain, many microservices.

Ingress Components

1. **Ingress Resource (YAML):** Defines the routing rules (what goes where)
2. **Ingress Controller:** The actual component that reads those rules and implements them. In Minikube, this is an Nginx Ingress Controller pod.

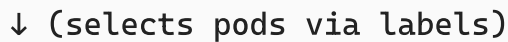
User Browser



Ingress Controller (NGINX pod in ingress-nginx namespace)



Kubernetes Services



Application Pods

Key Insight: Ingress does NOT talk directly to pods — it routes via Services. This preserves all the load-balancing and discovery benefits of Services.

Ingress Traffic Flow

Browser → `http://merndemo.local/api/tasks`



Host header: `merndemo.local`



Ingress Controller (NGINX)



Rule match: `path="/api"` → backend Service port 5000



Backend Service (ClusterIP, load-balances across pods)



Backend Pod (Node.js app handles the request)



Response flows back: Pod → Service → Ingress → Browser

Ingress YAML (`k8s/04-ingress.yaml`)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mern-ingress
  namespace: devops-part3
  annotations:
    # Strips the matched path prefix before forwarding to the
    service
    nginx.ingress.kubernetes.io/rewrite-target: /
    # Increase body size for file uploads
    nginx.ingress.kubernetes.io/proxy-body-size: "10m"
spec:
  ingressClassName: nginx      # Use the NGINX Ingress Controller
```

```

rules:
  - host: merndemo.local      # Custom domain (mapped via
    /etc/hosts for local dev)
    http:
      paths:
        # API requests go to backend service
        - path: /api
          pathType: Prefix    # Matches /api, /api/tasks,
                                /api/anything
          backend:
            service:
              name: backend
              port:
                number: 5000
        # All other requests go to frontend service
        - path: /
          pathType: Prefix
          backend:
            service:
              name: frontend
              port:
                number: 5173

```

Setting Up Ingress on Minikube

```

# Step 1: Enable the Nginx Ingress Controller addon
minikube addons enable ingress

# Step 2: Verify the controller is running
kubectl get pods -n ingress-nginx
# Expected: ingress-nginx-controller-xxxxx 1/1 Running

kubectl get all -n ingress-nginx
# Shows pod, service, and deployment for the controller

# Step 3: Apply your Ingress resource
kubectl apply -f k8s/04-ingress.yaml

# Step 4: Check the Ingress was created
kubectl -n devops-part3 get ingress
# NAME          CLASS  HOSTS          ADDRESS          PORTS
# mern-ingress  nginx  merndemo.local  192.168.49.2    80

```

```
# Step 5: Open minikube tunnel (KEEP THIS TERMINAL OPEN)
# Creates a bridge between localhost and the cluster – required on
Windows/Mac
minikube tunnel

# Step 6: Add hosts entry for local DNS resolution
# Windows: Edit C:\Windows\System32\drivers\etc\hosts (as
Administrator)
# Add: 127.0.0.1    merndemo.local
# Then flush DNS:
ipconfig /flushdns

# Step 7: Test
curl.exe -v http://merndemo.local/
curl.exe -v http://merndemo.local/api/tasks
# Or open in browser: http://merndemo.local/
```

Why minikube tunnel ? On Windows/Mac, Minikube runs inside a VM. The cluster's IP (192.168.49.x) is only reachable from within the VM. `minikube tunnel` creates a network bridge that maps `127.0.0.1:80` on your Windows machine directly into the Ingress Controller inside the cluster.

Frontend Rebuild for Ingress (Critical Step)

When using Ingress, the frontend must be built with `VITE_API_URL=/api` (relative path):

```
docker build -t frontend:local \
  -f client/Dockerfile \
  --build-arg VITE_API_URL=/api \
  client/

minikube image load frontend:local
kubectl -n devops-part3 rollout restart deploy/frontend
```

Why? With Ingress, the browser and the Ingress Controller are on the same origin (`merndemo.local`). The browser calls `/api/tasks` (relative URL) → becomes `http://merndemo.local/api/tasks` → Ingress routes to backend. No CORS, no cross-origin issues.

Storage & Persistence in Kubernetes (PVC)

The emptyDir Problem

In development, MongoDB deployments often use `emptyDir` for storage:

```
volumes:
  - name: mongo-data
    emptyDir: {}    # Temporary storage – GONE when pod is deleted
                    or rescheduled
```

This is like a student keeping notes only in RAM — every time the class ends (pod restarts), everything is gone.

Kubernetes Storage Concepts

PersistentVolume (PV): A piece of storage in the cluster, provisioned by an admin or dynamically via StorageClass. Independent of any pod lifecycle. Like a school locker — it exists regardless of which student (pod) is using it.

PersistentVolumeClaim (PVC): A request by a pod for storage. "I need a locker with at least 1GB space." K8s matches the PVC to an available PV and binds them.

StorageClass: Defines *how* storage is provisioned (SSD vs HDD, cloud EBS vs local disk). In Minikube, the default StorageClass auto-provisions PVs for you.

```
PVC (the request)
  ↓ Kubernetes matches
PV (the actual storage)
  ↓ Mounted into
Pod's container (/data/db)
```

PVC Definition (`k8s/05-mongo-pvc.yaml`)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongo-pvc
  namespace: devops-part3
spec:
  accessModes:
```

```
- ReadWriteOnce      # Can be mounted read/write by a single node
at a time
resources:
  requests:
    storage: 1Gi      # Request 1GB of storage
```

Access Modes:

- ReadWriteOnce (RWX) — Read/write by a single node (most common, used for databases)
- ReadOnlyMany (ROX) — Read-only by many nodes
- ReadWriteMany (RWX) — Read/write by many nodes (requires NFS or cloud storage)

Updating Mongo Deployment to Use PVC

Replace `emptyDir` with:

```
volumes:
  - name: mongo-data
    persistentVolumeClaim:
      claimName: mongo-pvc      # Reference the PVC by name

volumeMounts:
  - name: mongo-data
    mountPath: /data/db        # MongoDB's data directory
```

Demo: Proving Persistence

```
# Step 1: Apply PVC and updated Mongo deployment
kubectl apply -f k8s/05-mongo-pvc.yaml
kubectl apply -f k8s/01-mongo.yaml
kubectl -n devops-part3 rollout restart deploy/mongo
kubectl -n devops-part3 rollout status deploy/mongo
```

```
# Step 2: Verify PVC is bound
kubectl -n devops-part3 get pvc
```

# NAME	STATUS	VOLUME	CAPACITY	STORAGECLASS	AGE
# mongo-pvc	Bound	pvc-xxx	1Gi	standard	30s

```
# Step 3: Create some data via the frontend
# Open http://merndemo.local → create a task
```

```
# Step 4: Delete the Mongo pod (simulates crash/reschedule)
kubectl -n devops-part3 delete pod -l app=mongo

# Step 5: Watch it come back
kubectl -n devops-part3 get pods -w

# Step 6: Verify data survived
# Open http://merndemo.local → task is still there!
```

Complete MERN on Kubernetes: All YAML Files

k8s/00-namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: devops-part3
```

k8s/01-mongo.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mongo
  namespace: devops-part3
  labels:
    app: mongo
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mongo
  template:
    metadata:
      labels:
        app: mongo
    spec:
      containers:
        - name: mongo
          image: mongo:6.0
```

```

    ports:
      - containerPort: 27017
    volumeMounts:
      - name: mongo-data
        mountPath: /data/db
    volumes:
      - name: mongo-data
        persistentVolumeClaim:
          claimName: mongo-pvc      # Use PVC for production
persistence
---
apiVersion: v1
kind: Service
metadata:
  name: mongo
  namespace: devops-part3
spec:
  selector:
    app: mongo
  ports:
    - port: 27017
      targetPort: 27017
  type: ClusterIP      # Only accessible inside the cluster

```

k8s/02-backend.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: devops-part3
  labels:
    app: backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:

```

```

containers:
  - name: backend
    image: backend:local
    imagePullPolicy: IfNotPresent
    ports:
      - containerPort: 5000
    env:
      - name: MONGO_URI
        value: "mongodb://mongo:27017/taskdb"
    readinessProbe:
      httpGet:
        path: /health
        port: 5000
      initialDelaySeconds: 3
      periodSeconds: 5
      failureThreshold: 3
    livenessProbe:
      httpGet:
        path: /health
        port: 5000
      initialDelaySeconds: 10
      periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: backend
  namespace: devops-part3
spec:
  selector:
    app: backend
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
  type: ClusterIP

```

k8s/03-frontend.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend

```



```

    namespace: devops-part3
    labels:
      app: frontend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: frontend:local
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5173
---
apiVersion: v1
kind: Service
metadata:
  name: frontend
  namespace: devops-part3
spec:
  selector:
    app: frontend
  ports:
    - port: 5173
      targetPort: 5173
  type: ClusterIP

```

k8s/04-ingress.yaml

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mern-ingress
  namespace: devops-part3
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:

```

```

ingressClassName: nginx
rules:
  - host: merndemo.local
    http:
      paths:
        - path: /api
          pathType: Prefix
          backend:
            service:
              name: backend
              port:
                number: 5000
        - path: /
          pathType: Prefix
          backend:
            service:
              name: frontend
              port:
                number: 5173

```

k8s/05-mongo-pvc.yaml

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongo-pvc
  namespace: devops-part3
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi

```

kubectl Quick-Reference Table

Command	Use Case
kubectl get pods	List all pods in the default namespace
kubectl get pods -n <ns>	List pods in a specific namespace

Command	Use Case
<code>kubectl get pods -o wide</code>	List pods with additional details (node, IP)
<code>kubectl get pods -w</code>	Watch pods in real-time (live updates)
<code>kubectl get pods -l app=backend</code>	List pods filtered by label
<code>kubectl get deployments -n <ns></code>	List all deployments in a namespace
<code>kubectl get svc -n <ns></code>	List all services in a namespace
<code>kubectl get pods,svc -o wide -n <ns></code>	List pods AND services together
<code>kubectl get ingress -n <ns></code>	List all Ingress resources
<code>kubectl get pvc -n <ns></code>	List all PersistentVolumeClaims
<code>kubectl get configmap -n <ns></code>	List all ConfigMaps
<code>kubectl get secret -n <ns></code>	List all Secrets
<code>kubectl get nodes</code>	List all cluster nodes and their status
<code>kubectl get all -n <ns></code>	List all resources in a namespace
<code>kubectl apply -f <file>.yaml</code>	Create or update resources from a YAML file (declarative)
<code>kubectl apply -f k8s/</code>	Apply all YAML files in a directory
<code>kubectl delete -f <file>.yaml</code>	Delete resources defined in a YAML file
<code>kubectl delete namespace <ns></code>	Delete a namespace and ALL resources inside it
<code>kubectl describe pod <name> -n <ns></code>	Show detailed info about a pod including events — use for debugging
<code>kubectl describe svc <name> -n <ns></code>	Show service details including endpoints (confirms pods are selected)
<code>kubectl logs <pod> -n <ns></code>	View stdout/stderr logs of a pod
<code>kubectl logs -l app=backend -n <ns></code>	View logs from all pods with a label
<code>kubectl logs <pod> --tail=200 -n <ns></code>	View last 200 lines of logs
<code>kubectl exec -it <pod> -n <ns> -- bash</code>	Open interactive shell inside a running pod

Command	Use Case
<code>kubectl exec -it <pod> -n <ns> -- env</code>	Print environment variables inside a pod
<code>kubectl port-forward svc/<svc> 5000:5000 -n <ns></code>	Forward local port to a service port (testing only)
<code>kubectl rollout restart deploy/<name> -n <ns></code>	Rolling restart of a deployment (pick up new config/image)
<code>kubectl rollout status deploy/<name> -n <ns></code>	Watch the status of a rolling update
<code>kubectl rollout undo deploy/<name> -n <ns></code>	Rollback to the previous deployment version
<code>kubectl scale deploy/<name> -- replicas=3 -n <ns></code>	Manually scale a deployment
<code>kubectl set image deploy/<name> <container>=<image> -n <ns></code>	Update the image of a deployment (imperative)
<code>kubectl -n <ns> wait -- for=condition=ready pod -l app=mongo --timeout=180s</code>	Block until pods with label are ready — use in scripts
<code>kubectl -n <ns> get endpoints <svc></code>	Show which pod IPs a service is routing to — diagnose no-endpoint issues
<code>minikube start --driver=docker</code>	Start Minikube using Docker as the VM driver
<code>minikube addons enable ingress</code>	Enable the NGINX Ingress Controller in Minikube
<code>minikube tunnel</code>	Create network tunnel for Ingress access on Windows/Mac
<code>minikube ip</code>	Get the Minikube cluster IP
<code>minikube image load <image></code>	Load a local Docker image into the Minikube node (no registry needed)
<code>minikube image ls</code>	List all images loaded into Minikube
<code>minikube image rm <image></code>	Remove an image from Minikube's image store

Complete Run Order (MERN on Minikube — From Scratch)

Section A: Cleanup

```
# Delete the namespace (quickest full cleanup)
kubectl delete namespace devops-part3

# Wait until namespace is fully removed (PowerShell)
while (kubectl get namespace devops-part3 -o json 2>$null) {
    Write-Host "Waiting for namespace to terminate..."
    Start-Sleep -Seconds 3
}

# Remove images from Minikube's store
minikube image rm backend:local 2>$null
minikube image rm frontend:local 2>$null

# Optionally remove from host Docker too
docker rmi backend:local frontend:local 2>$null
```

Section B: Build and Load

```
# Build backend image
docker build -t backend:local -f server/Dockerfile server/

# Build frontend with VITE_API_URL for Ingress setup
docker build -t frontend:local \
    -f client/Dockerfile \
    --build-arg VITE_API_URL=/api \
    client/

# Load into Minikube (skip registry push for local dev)
minikube image load backend:local
minikube image load frontend:local

# Verify
minikube image ls | grep backend
minikube image ls | grep frontend
```

Section C: Apply Manifests

```
# Create namespace
kubectl apply -f k8s/00-namespace.yaml
```

```
# Apply ConfigMap and Secret
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/secret-mongo.yaml

# Apply PVC (before Mongo, so storage is ready)
kubectl apply -f k8s/05-mongo-pvc.yaml

# Deploy MongoDB and wait for it to be ready
kubectl apply -f k8s/01-mongo.yaml
kubectl -n devops-part3 wait --for=condition=ready pod -l app=mongo
--timeout=180s
kubectl -n devops-part3 get pods -l app=mongo -o wide

# Deploy backend and wait
kubectl apply -f k8s/02-backend.yaml
kubectl -n devops-part3 wait --for=condition=ready pod -l
app=backend --timeout=180s
kubectl -n devops-part3 get pods -l app=backend -o wide
kubectl -n devops-part3 get svc backend -o wide

# Deploy frontend and wait
kubectl apply -f k8s/03-frontend.yaml
kubectl -n devops-part3 wait --for=condition=ready pod -l
app=frontend --timeout=180s

# Enable Ingress and apply rules
minikube addons enable ingress
kubectl apply -f k8s/04-ingress.yaml
```

Section D: Tunnel and Test

```
# Terminal A: Keep minikube tunnel running
minikube tunnel

# Edit hosts file (Windows - as Administrator)
# C:\Windows\System32\drivers\etc\hosts
# Add: 127.0.0.1 merndemo.local
ipconfig /flushdns

# Terminal B: Test
curl.exe -v http://merndemo.local/
curl.exe -v http://merndemo.local/api/tasks
```

```
# View logs
```

```
kubectl -n devops-part3 logs -l app=backend --tail=200
```

```
# Check all pods and services
```

```
kubectl -n devops-part3 get pods,svc -o wide
```

Troubleshooting Guide

Symptom	Likely Cause	Fix
"Could not resolve host"	Containers not on the same user-defined network	Verify networks: in docker-compose.yml; add both containers to same network
ECONNREFUSED	Target container not listening on that port, wrong port, or wrong service name	Check containerPort, targetPort, and service selector labels
Service has no endpoints	Label mismatch between Service selector and pod template labels	kubectl -n <ns> get endpoints <svc> → verify app: labels match exactly
Frontend can't reach backend	Using absolute URL (http://backend:5000) in production with Ingress	Rebuild frontend with VITE_API_URL=/api
Permission denied on Docker socket in Jenkins	Jenkins user not in Docker group	docker exec -u 0 jenkins chmod 666 /var/run/docker.sock
Minikube Ingress connection refused on apply	API server not ready when addon YAML applies	Rerun minikube addons enable ingress after minikube status shows Running
Pod stuck in CrashLoopBackOff	Container crashes immediately on start	kubectl describe pod <name> → check Events; kubectl logs <pod> → check error
ImagePullBackOff	Kubernetes can't pull the image	Verify imagePullPolicy: IfNotPresent for local images; run minikube image load

Symptom	Likely Cause	Fix
PVC stuck in Pending	No PV available to satisfy the claim	Check StorageClass: <code>kubectl get storageclass</code> ; in Minikube the standard class auto-provisions
Data lost after pod restart	Using <code>emptyDir</code> instead of PVC	Replace <code>emptyDir: {}</code> with <code>persistentVolumeClaim:</code> <code>claimName: mongo-pvc</code>

Interview Edge: Kubernetes Deep Questions

Q: "Why choose Ingress over NodePort in production?" A: NodePort exposes a service on a random high port (30000-32767) on every cluster node. For 10 microservices, that's 10 different ports users must know. Ingress provides a single entry point on port 80/443, routes traffic based on HTTP host/path rules (Layer 7), supports SSL termination, and integrates with DNS — all without exposing individual service ports. NodePort is only suitable for testing or single-service internal tools.

Q: "How does K8s self-healing actually trigger?" A: The Controller Manager runs a continuous **reconciliation loop** — it constantly compares the *desired state* (stored in etcd) with the *actual state* (polled from kubelets). If a pod crashes, the kubelet on that node reports the pod is gone. The Controller Manager detects the gap: "desired=3, actual=2." It creates a new pod spec and passes it to the Scheduler, which picks a node. The kubelet on that node starts the container. This entire loop typically completes in seconds.

Q: "What's the difference between a Deployment and a ReplicaSet?" A: A ReplicaSet ensures N replicas of a pod are running. A Deployment is a higher-level wrapper around ReplicaSets that adds rolling updates and rollbacks. When you update a Deployment's image, it creates a NEW ReplicaSet with the new configuration and gradually scales it up while scaling down the old one. If the rollout fails, you can run `kubectl rollout undo` to revert to the previous ReplicaSet. You should almost never create ReplicaSets directly — always use Deployments.

Q: "Why are pods ephemeral, and why does that matter?" A: Pods don't have stable identities by design. When a pod dies (crash, node failure, update), a new pod is created — it gets a new name, a new IP address, and potentially runs on a different node. This is why direct pod-to-pod communication is unreliable. Services solve this by providing a stable virtual IP and DNS name that always routes to currently healthy pods, regardless of which specific pod instances are alive.

Q: "What is the difference between a readiness probe and a liveness probe?" A: Readiness controls *traffic routing* — if the readiness probe fails, the pod is removed from the Service's endpoint list (no traffic sent to it), but the pod itself keeps running. This prevents routing requests to a container that hasn't fully started yet (e.g., hasn't connected to the database). Liveness controls *pod lifecycle* — if the liveness probe fails, kubelet kills and restarts the container. This handles processes that are "stuck" or deadlocked.

Q: "How does K8s networking differ from Docker Compose networking?" A: Both use DNS-based service discovery. In Docker Compose, the DNS name is the service/container name within a custom bridge network. In K8s, the DNS name is the Service object's name, and it resolves to the Service's ClusterIP — not directly to any pod. K8s DNS also provides fully-qualified names: `backend.devops-part3.svc.cluster.local`, though short names like `backend` work within the same namespace. The critical difference: K8s Services provide load balancing across multiple pod replicas, while Docker Compose services also load-balance but within a single machine.

Q: "What is the Reconciliation Loop and why is it fundamental to K8s?" A: The Reconciliation Loop is the core operational philosophy of Kubernetes. Every controller (Deployment controller, ReplicaSet controller, etc.) runs an infinite loop: read current state → compare to desired state → take actions to close the gap → repeat. This is what makes K8s self-healing: you don't need to tell K8s "restart that crashed pod" — it continuously detects the gap and corrects it automatically. It also enables GitOps: change the desired state in a YAML file, K8s reconciles the cluster to match.

Q: "Why do we need both ConfigMaps AND Secrets? Why not just use Secrets for everything?" A: ConfigMaps are designed for non-sensitive configuration — API URLs, feature flags, hostnames. They're stored as plain text and are easy to read and audit. Secrets are for sensitive data (passwords, tokens, certificates) and are stored as base64-encoded values with additional access controls (RBAC can restrict who can read secrets). Using Secrets for non-

sensitive config adds unnecessary complexity and access overhead. Using ConfigMaps for sensitive data is a security risk. The separation enforces clarity about what needs protection.

Summary: The Full Evolution

Phase 1: Write code → Git → GitHub

↓

Phase 2: Docker packages it → Docker Compose runs it locally

↓

Phase 3: GitHub Actions tests it → Deploys it to EC2 via SSH
OR: Jenkins builds it, tests it, pushes to registry, deploys

↓

Phase 4: Nginx routes traffic intelligently (one entry point)
Kubernetes manages it at scale (self-healing, auto-scaling, persistence)

The Evolution of "Where Does Nginx Live?"

1. **Phase 2 (Docker Compose):** Nginx as a container in docker-compose.yml, routes to other containers via Docker DNS
2. **Phase 4 (Kubernetes):** Nginx Ingress Controller as a K8s pod in `ingress-nginx` namespace, routes to Services via K8s DNS

The Evolution of "Where Does My Data Live?"

1. **Phase 2:** Docker named volume on the EC2 host filesystem
2. **Phase 4:** Kubernetes PersistentVolumeClaim → PersistentVolume (dynamically provisioned by StorageClass)

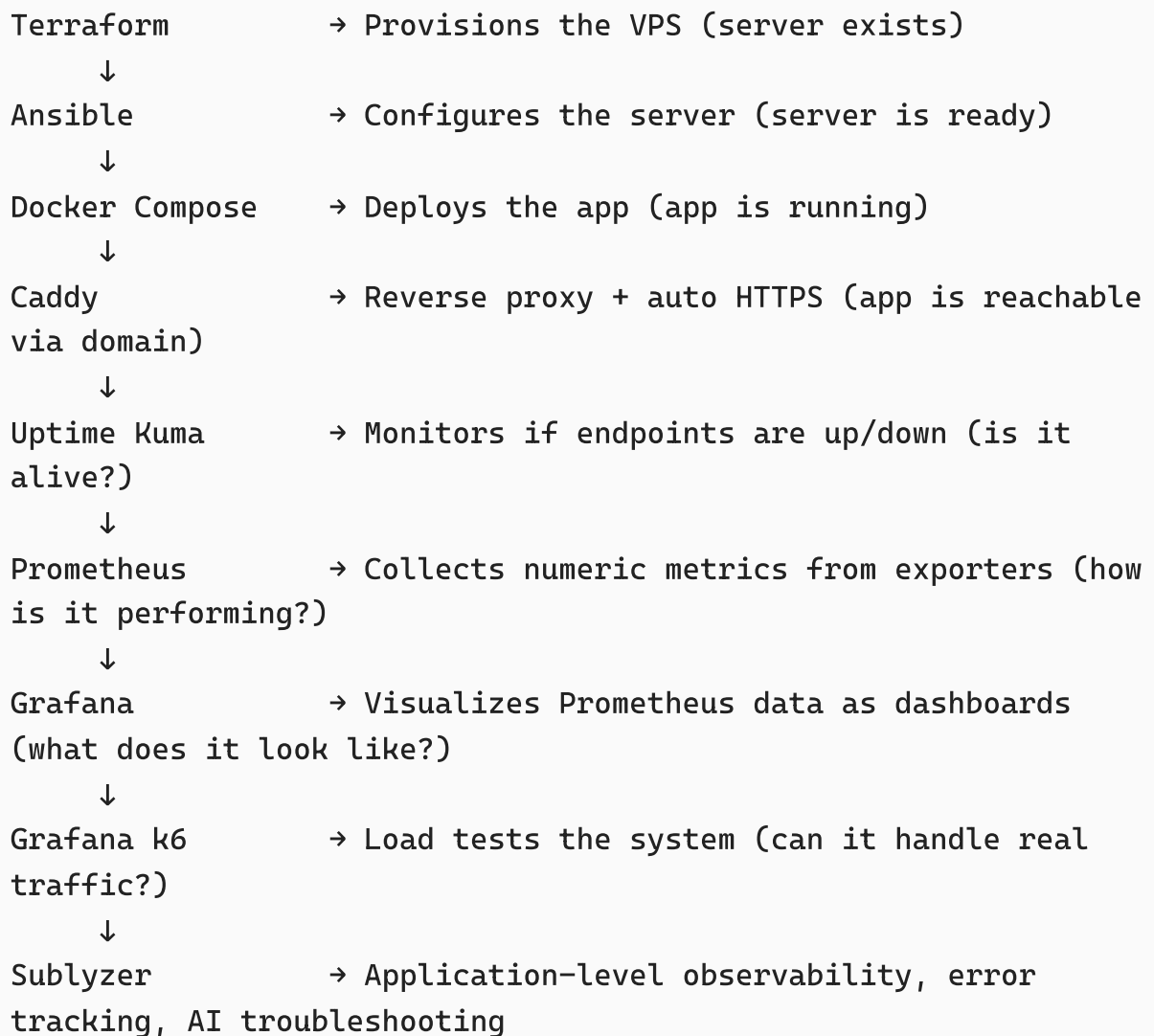
The Evolution of "How Does Service Discovery Work?"

1. **Docker Compose:** Container names = DNS names within a user-defined bridge
 2. **Kubernetes:** Service object names = DNS names within a namespace
(`backend.devops-part3.svc.cluster.local`)
-
-

PHASE 5: PRODUCTION INFRASTRUCTURE

Terraform, Ansible, Caddy, and the Full Observability Stack

The Full Production Pipeline:



5.1 — Tools Overview & Project Architecture

A production system needs more than just running containers on a server. Here is what each tool handles and exactly why it exists in the stack:

Tool	Layer	What It Does
Terraform	Infrastructure	Creates/destroys the VPS, DNS records, SSH keys via cloud provider APIs
Ansible	Configuration	Installs Docker, configures the OS, deploys apps — turns a raw server into a ready target
Caddy	Edge / Networking	Reverse proxy on ports 80/443, handles automatic HTTPS via Let's Encrypt
Uptime Kuma	Monitoring — Availability	Checks if your endpoints are UP or DOWN, shows response time graphs, sends alerts
Prometheus	Monitoring — Metrics	Scrapes and stores time-series numeric data from exporters every 15 seconds
Grafana	Monitoring — Visualization	Connects to Prometheus and renders metrics as charts, graphs, and dashboards
Grafana k6	Testing	Scripted load testing — simulates concurrent users to expose performance limits
Sublyzer	Observability	Real-time error tracking, performance monitoring, and AI-assisted troubleshooting
Hostinger VPS	Hosting	The Virtual Private Server where the application runs in production

The Critical Distinction — Provisioning vs. Configuration Management:

This is one of the most common DevOps interview questions:

- **Provisioning (Terraform):** Focuses on *creating, changing, or destroying infrastructure*. It answers the question: "What should exist?" This includes servers, networks, DNS records, and load balancers. Terraform talks to **cloud provider APIs** (Hostinger, AWS, GCP) and maintains the **state** of those resources.
- **Configuration Management (Ansible):** Focuses on *turning a provisioned server into a usable deployment target*. It answers the question: "What should be installed and configured on the server?" Ansible talks directly to **the server over SSH** and runs tasks like installing Docker, writing config files, and starting services.

The output of Terraform (such as a VPS IP address) becomes the **input** for Ansible.

5.2 — Terraform: Infrastructure as Code (IaC)

What & Why

What Terraform is: A tool for **Infrastructure as Code (IaC)** — you define your entire infrastructure (servers, networks, DNS) in `.tf` configuration files, and Terraform manages the full lifecycle of those resources: creating, updating, and destroying them via cloud provider APIs.

The Old Way (The Problem): Before IaC, provisioning a server meant clicking through a cloud console, manually copying settings, and hoping you remembered to configure everything the same way in dev, staging, and production. This created **snowflake servers** — unique configurations that were impossible to reproduce.

What Terraform Solves:

- **Reproducibility:** Spin up identical environments in dev, staging, and prod from the same code
- **Version control:** Infrastructure changes are tracked in Git, reviewable, and reversible
- **Automation:** CI/CD pipelines can provision and tear down infrastructure automatically
- **Auditability:** The state file records exactly what infrastructure exists and who changed it

The Terraform CLI Workflow

The CLI is the central interface that reads your `.tf` files, interacts with provider APIs, and executes infrastructure changes. Think of it as the bridge between your local machine (or CI/CD runner) and the cloud provider.

```
Your .tf files (desired state)
```

```
↓
```

```
Terraform CLI reads files → interacts with Provider APIs
```

```
↓
```

- | | |
|-----------------------|---|
| 1. terraform init | → Downloads provider plugins, prepares directory |
| 2. terraform fmt | → Auto-formats code to standard style |
| 3. terraform validate | → Checks syntax and logic (no API calls) |
| 4. terraform plan | → Calculates the diff (desired vs. current state) |
| 5. terraform apply | → Applies changes via provider APIs |
| 6. terraform output | → Prints defined output values (e.g., VPS IP) |

The 6 Core Commands Explained:

terraform init The mandatory first step. Initializes the working directory by downloading the necessary provider plugins required to interact with your cloud or infrastructure services. Creates the `.terraform.lock.hcl` file (analogous to `package-lock.json` in Node.js — ensures every team member uses the exact same provider version).

```
terraform init
# Output: Initializing provider plugins...
# Creates: .terraform/ folder and .terraform.lock.hcl
```

terraform fmt Automatically formats your `.tf` files to follow standard Terraform style guidelines, ensuring consistency across the team.

```
terraform fmt
```

terraform validate Performs a validation check on your configuration files to ensure the syntax is correct and the logic is valid before you attempt any deployments. Does NOT call any APIs.

```
terraform validate
# Output: Success! The configuration is valid.
```

terraform plan The most important safety step. Generates a "diff" — shows you exactly what will be **created (+)**, **modified (~)**, or **destroyed (-)** without actually making any changes. Always review this output carefully before applying.

```
terraform plan
# Sample output:
```

```
# + resource "hostinger_vps" "prod" { ← will be created
# ~ hostname = "old-name" -> "new-name" ← will be changed
# - resource "hostinger_vps" "staging" ← will be destroyed
```

terraform apply Executes the plan and deploys/modifies infrastructure. Asks for confirmation (type `yes`) unless you pass `-auto-approve` .

```
terraform apply
terraform apply -auto-approve # Skip the "yes" confirmation (for
CI/CD)
```

terraform output After infrastructure is provisioned, prints the defined output values — like the VPS IP address — which are often required as input for downstream tools like Ansible.

```
terraform output
# vps_ip = "187.127.157.4"
# vps_id = "1618885"
```

The Terraform Block Structure

Infrastructure in Terraform is defined through specific structural blocks:

<code>terraform block</code>	→ Meta settings: versions to lock & which providers are needed
<code>provider block</code>	→ How to authenticate with the target platform (API tokens)
<code>resource block</code>	→ The actual infra objects to manage (VPS, DNS, networks)
<code>variable block</code>	→ Inputs: values that can be customized per environment
<code>output block</code>	→ Results: information to expose after deployment
<code>data block</code>	→ Read-only lookups of existing infrastructure (e.g., list data centers)

Visual representation:

terraform block → meta settings → versions (lock) & providers
project rules + what plugins I need

provider block → how to authenticate / configure that provider
tell terraform how to talk to the provider api

```
provider "hostinger" {  
    api_token = var.hostinger_api_token  
}
```

resource block → actual infra object

```
resource "hostinger_vps" "prod" {  
    ...  
}
```

variable block → inputs

```
variable "hostname" {  
    type          = string  
    description = "this is my hostname"  
}
```

output block → print/provide to downstream tools

Terraform File Structure (Professional Standard)

```
ops/  
└─ terraform/  
    └─ prod/                                ← Environment folder  
        └─ main.tf                          ← Core resource definitions (the  
"what")  
            └─ variables.tf                 ← Variable declarations (the  
"shape")  
                └─ outputs.tf               ← Output definitions (the  
"return values")  
                    └─ terraform.tfvars     ← Variable values (the "data") –  
gitignored!  
                        └─ terraform.tfvars.example ← Documentation template –  
safe to commit  
                            └─ .terraform.lock.hcl ← Provider version lock – commit  
this
```


└─ terraform.tfstate ← State file – NEVER commit, use remote backend

File-by-File Breakdown

main.tf — The Primary Entry Point

Contains the core resource definitions, provider configurations, and module calls. Think of it as the **blueprint of the building**.

```
terraform {
  required_providers {
    hostinger = {
      source  = "hostinger/hostinger"
      version = "0.1.22"    # Pin to specific version to avoid
                             surprise breakage
    }
  }
}

provider "hostinger" {
  api_token = var.hostinger_api_token    # Never hardcode tokens
  here!
}

# Data source: read-only lookup of available data centers
data "hostinger_vps_data_centers" "all" {}
data "hostinger_vps_templates" "all" {}
data "hostinger_vps_plans" "all" {}

# Resource: the actual VPS to create/manage
resource "hostinger_vps" "prod" {
  plan           = var.plan
  data_center_id = var.data_center_id
  template_id    = var.template_id
  hostname       = var.hostname
  password       = var.root_password
}
```

Best practices for main.tf :

- If it exceeds ~150 lines, split resources into logically named files (`network.tf` , `iam.tf`)
- Keep it focused on related resources — avoid managing completely unrelated systems in one file ("Terralith" anti-pattern)
- Keep variables and outputs in their own separate files — `main.tf` should only contain logic

`variables.tf` — Input Declarations

This file **declares** variables — it defines their name, type, and optionally a description and default value. It does NOT assign values.

```
# Required variable (no default = Terraform FORCES user to provide it)
variable "hostinger_api_token" {
    type          = string
    description   = "API token from Hostinger control panel"
    sensitive     = true      # Prevents value from being printed in CLI
output
}

# Optional variable (has a default = acts as fallback if not provided)
variable "hostname" {
    type          = string
    description   = "Hostname for the VPS"
    default       = "prod-server-1"
}

variable "plan" {
    type          = string
    default       = "hostingerin-vps-kvm1-inr-1m"
}

variable "data_center_id" {
    type          = number
    default       = 13      # Mumbai data center
}
```

Variable types supported:

- `string` — Text: `"us-east-1"`

- `number` — Numeric: `3` , `5.5`
- `bool` — True/false: `true` , `false`
- `list` — Ordered collection: `["a", "b", "c"]`
- `map` — Key-value pairs: `{ env = "prod", region = "us" }`

Required vs. Optional:

- Declare a variable **without** `default` → it becomes **required** (Terraform will error if not provided)
- Add `default = "value"` → it becomes **optional** (falls back to the default)

outputs.tf — The "Return Values" of Your Infrastructure

Outputs serve three purposes:

1. **CLI Feedback** — Display important data (like VPS IP) after `terraform apply`
2. **Inter-Module Communication** — Pass data from child modules to parent modules
3. **Downstream Input** — The VPS IP output becomes Ansible's inventory input

```
output "vps_ip" {
  description = "The public IPv4 address of the production VPS"
  value       = hostinger_vps.prod.ipv4_address
}

output "vps_id" {
  description = "The Hostinger VPS resource ID"
  value       = hostinger_vps.prod.vps_id
}
```

terraform.tfvars — Variable Assignment

This file **assigns** actual values to the variables declared in `variables.tf`.

Terraform automatically loads this file (or `*.auto.tfvars`) without you specifying it on the command line.

```
# terraform.tfvars - NEVER commit this to Git
plan           = "hostingerin-vps-kvm1-inr-1m"
data_center_id = 13
template_id    = 1200
```

```
hostname      = "mern-prod-server"
root_password = "your_secure_password_here"
```

Why not hardcode values in `main.tf` ?

1. **Environment switching** — You can have `dev.tfvars` , `staging.tfvars` , and `prod.tfvars` . Switch environments by pointing Terraform to a different file: `terraform apply -var-file="prod.tfvars"`
2. **Security** — Add `*.tfvars` to `.gitignore` so sensitive values (account IDs, passwords) never reach your public GitHub repo

`terraform.tfvars.example` — Documentation Template (Safe to Commit)

A documentation-only file that shows other developers exactly which values they need to provide. Terraform does NOT read this file — it is purely for humans.

```
# terraform.tfvars.example
# Copy this file to terraform.tfvars and fill in your actual
# values.
# DO NOT commit terraform.tfvars - it contains sensitive data.

plan          = "your_plan_slug_here"
data_center_id = 0          # Get this from: terraform console >
data.hostinger_vps_data_centers.all
template_id   = 0          # Get this from: terraform console >
data.hostinger_vps_templates.all
hostname      = "your-server-hostname"
root_password = "your_secure_root_password"
```

The Two Critical State Files

`.terraform.lock.hcl` — The Dependency Lock File

Ensures that every team member uses the **exact same version** of provider plugins. Contains version numbers and cryptographic checksums (hashes) that verify the provider code hasn't been tampered with.

Analogy: `package-lock.json` in Node.js.

```
# .terraform.lock.hcl - COMMIT this file to Git
provider "registry.terraform.io/hostinger/hostinger" {
  version      = "0.1.22"
  constraints = "0.1.22"
  hashes = [
    "h1:abc123...", # Checksum for Linux AMD64
    "h1:def456...", # Checksum for macOS ARM64
  ]
}
```

terraform.tfstate — The "Brain" of Terraform

The most important file you've never edited directly. It is a JSON file that acts as the **single source of truth**, mapping your HCL code (`"web_server"`) to the real-world cloud resource (`i-0abc123def456`).

What it stores:

- **Mapping:** Your code names a resource `"prod"` , but Hostinger sees it as ID `1618885` . The state file remembers this relationship.
- **Metadata:** Resource dependencies, private IPs, generated resource IDs, and other attributes not written in your code but needed for updates.
- **Performance:** Instead of calling the cloud API to check every resource on every `plan` , Terraform consults the local state file first.

The State Lifecycle:

1. Empty: No state exists before first apply
2. `terraform apply`: Creates resources → writes their real-world IDs into `terraform.tfstate`
3. `terraform plan` (next time): Compares Code vs. State File vs. Reality → shows diff
4. `terraform destroy`: Deletes resources → clears them from `terraform.tfstate`

Critical Warnings:

- **Never edit this file manually.** If you need to fix state, use `terraform state` commands.
- **Sensitive data risk:** The state file contains everything in plain text, including passwords and secret keys generated during resource creation.

- **Never commit to public Git repos** — it exposes your entire infrastructure config.

Interview: Why is storing `terraform.tfstate` on a local laptop bad for a team?

Two reasons:

1. **Merge Conflicts / Split Brain:** If two engineers run `terraform apply` simultaneously, their local state files will diverge — each thinks they own different things, causing destructive conflicts.
2. **Security:** Local laptops are generally not encrypted to the standard needed for a file containing cloud credentials and resource IDs.

Solution: Use Remote Backends — store state centrally and safely:

- **AWS:** S3 bucket for state + DynamoDB table for state locking (prevents simultaneous applies)
- **Terraform Cloud / HCP Terraform:** Managed remote state with built-in locking and access controls

Environment Variables for Sensitive Inputs

Instead of storing the API token in `terraform.tfvars` (risky) or typing it every time `terraform plan` runs, you can set it as an OS environment variable. Terraform automatically picks up variables prefixed with `TF_VAR_`.

```
# Windows PowerShell
$env:TF_VAR_hostinger_api_token="your_token_here"

# macOS / Linux (Zsh/Bash)
export TF_VAR_hostinger_api_token="your_token_here"
```

Practical Workflow: Provisioning a Hostinger VPS

Here is the complete sequence used in the course, including a real import workflow for an existing VPS:

```
# Step 1: Clean up any stale state (if switching provider versions)
Remove-Item -Recurse -Force .terraform -ErrorAction
SilentlyContinue
Remove-Item -Force .terraform.lock.hcl -ErrorAction
SilentlyContinue

# Step 2: Initialize (downloads hostinger/hostinger v0.1.22
provider)
# Note: Previously v0.1.3 had issues; 0.1.22 was the fix
terraform init -upgrade

# Step 3: Format code
terraform fmt

# Step 4: Validate syntax
terraform validate
# Output: Success! The configuration is valid.

# Step 5: Set API token as environment variable
$env:TF_VAR_hostinger_api_token="your_token"

# Step 6 (if importing an EXISTING VPS instead of creating a new
one):
# terraform import tells Terraform: "This resource already exists -
start managing it"
terraform import hostinger_vps.prod 1618885
# Output: Import successful! Resources now in Terraform state.

# Step 7: See what Terraform would change
terraform plan
# Look for the symbols:
#   + = will create
#   ~ = will modify
#   - = will destroy
#   -/+ = destroy and recreate (forces replacement)

# Step 8: Apply (type "yes" when prompted)
terraform apply

# Step 9: Get the VPS IP for Ansible
terraform output
# vps_ip = "187.127.157.4"
```

Exploring available options with terraform console :

```
terraform console
> data.hostinger_vps_data_centers.all.data_centers
# Returns list of available data centers:
# {
#   "city" = "Mumbai"
#   "continent" = "Asia"
#   "id" = 13
#   "location" = "in"
#   "name" = "mum"
# }
```

Important version note: Provider version 0.1.3 had a known bug. The working version used in this course is 0.1.22 . Always check provider changelogs when something breaks after a version upgrade.

5.3 — Ansible: Configuration Management

What & Why

What Ansible is: An open-source IT automation engine that simplifies infrastructure management, software provisioning, and application deployment. It is the tool that turns a raw, freshly provisioned server into a fully configured, app-running environment.

The Core Purpose — Automation, Repeatability, and Confidence:

Without Ansible, configuring a server means:

```
ssh root@187.127.157.4
apt update
apt install docker.io
systemctl start docker
git clone https://github.com/user/app.git
cd app
docker compose up -d
# ... and so on, manually, every time
```


Every manual step introduces the risk of forgetting something, making a typo, or doing things slightly differently on the staging vs. production server. Ansible replaces this with a **YAML playbook** that you run once and it handles everything — repeatably and identically every time.

Ansible's role in the pipeline vs. Terraform:

```
Terraform → talks to cloud/provider APIs → manages all infra
resources
           keeps the state of resources → outputs values (e.g.,
IP) to downstream
```

```
Ansible → talks to the server directly over SSH → manages OS
config + deployments
         uses inventory to decide targets → uses playbooks to
describe automation
```

About SSH: Why We Need It First

Before understanding Ansible, it helps to understand SSH — because Ansible uses SSH as its transport layer.

SSH (Secure Shell) is a cryptographic protocol that allows you to connect to a remote machine securely and execute commands as if you were sitting at that machine's terminal. Every time you run `ssh root@187.127.157.4`, you authenticate (typically with an SSH key pair) and get a remote shell.

Why Ansible instead of just running SSH commands manually?

- **Repeatability:** SSH commands are one-time, manual. Ansible playbooks are reusable, documented automation.
- **Scalability:** Running the same SSH command on 50 servers manually is impractical. Ansible runs it on all 50 simultaneously.
- **Idempotency:** Ansible knows whether a task needs to run. If Docker is already installed, it won't reinstall it. SSH commands don't have this intelligence.
- **Documentation as code:** Playbooks serve as both automation and documentation — they tell you and your team exactly what was done to every server.

Core Ansible Concepts

Agentless Architecture

A major advantage of Ansible over competitors (like Chef, Puppet): **no software or "agents" need to be installed on target machines**. Ansible connects via standard SSH (Linux/Unix) or WinRM (Windows) to execute tasks, and then the modules it uses are removed after execution.

This makes setup dramatically simpler — you just need SSH access to the target server.

Control Node vs. Managed Nodes

- **Control Node:** The machine where Ansible is installed and from which you run playbooks. In our setup, this is your laptop (running WSL) or a CI/CD runner.
- **Managed Nodes (Hosts):** The remote systems being automated — in our case, the Hostinger VPS provisioned by Terraform.

```
Your Laptop (Control Node)
      ↓ (SSH connection)
VPS - 187.127.157.4 (Managed Node)
```

Inventory

An inventory is a list of managed nodes (servers) that Ansible knows about. Instead of hardcoding IP addresses into every playbook, you define them centrally in the inventory, organized into groups.

```
# inventory/hosts (inventory.ini format)
[prod]
187.127.157.4 ansible_user=root
ansible_ssh_private_key_file=~/.ssh/id_ed25519

[stage]
192.168.1.100 ansible_user=ubuntu

[monitoring]
187.127.157.4    # Same VPS can be in multiple groups
```

Why groups matter: You can run a playbook against only `prod` , only `stage` , or all servers at once. This prevents accidentally running a destructive operation on the wrong environment.

Playbooks

Human-readable YAML files that define the desired state of a system. They act as blueprints for automation — a sequence of tasks Ansible will execute on the target nodes.

```
# bootstrap.yml - Makes a raw server ready
---
- name: Bootstrap production server
  hosts: prod          # Target the "prod" group from inventory
  become: true         # Run as root (privilege escalation via
                        sudo)

  tasks:
    - name: Update apt package cache
      apt:
        update_cache: yes

    - name: Install Docker
      apt:
        name: docker.io
        state: present  # "present" = install if not there
                        (idempotent!)

    - name: Start and enable Docker service
      service:
        name: docker
        state: started
        enabled: yes

    - name: Install Docker Compose plugin
      get_url:
        url:
          "https://github.com/docker/compose/releases/download/v2.29.2/docker-
          compose-linux-x86_64"
        dest: /usr/local/lib/docker/cli-plugins/docker-compose
        mode: '0755'
```

Modules

Modules are the "tools" in Ansible's toolbox — small programs pushed to managed nodes to perform specific tasks (install packages, copy files, manage services, run shell commands). They are designed to be **idempotent**: they only make changes if the system is not already in the desired state.

Common modules:

Module	What It Does
apt / yum	Install/remove packages on Debian/RHEL systems
copy	Copy files from control node to managed node
template	Copy a Jinja2 template file (with variable substitution)
service	Start, stop, restart, or enable system services
shell / command	Run arbitrary shell commands
get_url	Download files from URLs
file	Create directories, set permissions, manage symlinks
lineinfile	Ensure a specific line exists in a file
docker_compose	Manage Docker Compose projects

Idempotency: The Core Principle

Running the same Ansible playbook 10 times should produce the same result as running it once, without breaking anything. This is **idempotency** — and it is what makes Ansible safe to run in CI/CD pipelines and as part of automated deployments.

become: true — Privilege Escalation

```
become: true          # Default: sudo to root
become_method: sudo   # The escalation method (sudo, su, pbrun, etc.)
become_user: root      # The target user to become
```

This is essential for tasks that require elevated permissions: installing packages, modifying system files, creating users, managing services.

Ansible Output: Understanding Task Status

After every task, Ansible reports one of three statuses:

Status	Color	Action Taken?	Meaning
ok	Green	No	Task finished successfully; system was already in the desired state
changed	Yellow	Yes	Task finished successfully and a modification was made
failed	Red	No/Partial	An error occurred; Ansible stops further tasks on this host

Why ok is the goal: An `ok` result means your server was already in the desired state — this is idempotency working correctly. If you re-run a fully converged playbook and get all `ok`, everything is healthy.

Practical tips:

- `ignore_errors: true` — Allows the playbook to continue even if a specific task fails (use carefully)
- Handlers only trigger on `changed` — if a config file didn't change, the "restart nginx" handler won't fire unnecessarily

Project Directory Structure (Professional Standard)

```
ops/
├── ansible/
│   ├── ansible.cfg                # Project-specific configuration
│   (prevents extra flags)
│   ├── inventory/
│   │   ├── hosts                 # Default inventory (or
inventory.ini)
│   │   ├── group_vars/
│   │   │   ├── all.yml          # Variables applied to ALL groups
│   │   │   └── host_vars/
│   │   │       └── 187.127.157.4.yml # Variables for a specific host
│   ├── playbooks/
│   │   ├── bootstrap.yml        # Makes raw server ready (install
Docker, Git, etc.)
│   │   └── deploy.yml           # Deploys the application
```

```

├── observability.yml    # Deploys the monitoring stack
└── roles/               # Reusable, modular automation logic
    ├── common/
    │   ├── tasks/      # Main task logic
    │   └── handlers/   # Service restarts (trigger on
    │                       "changed")
    ├── templates/      # Jinja2 templates (.j2) with
    │                       variable substitution
    └── files/          # Static files to copy to managed
nodes
    ├── vars/           # Role-specific variables
    ├── webserver/
    └── ...

```

Why this structure matters:

- `ansible.cfg` — Stores project settings like inventory path, SSH key location, and remote user, so you don't pass `--inventory`, `--user` flags on every command
- `group_vars/all.yml` — Global variables available to all hosts (e.g., `docker_version`, `app_dir`)
- `host_vars/` — Per-host overrides (e.g., one server has a different port)
- Separating `playbooks/` from `roles/` — Playbooks orchestrate; roles provide reusable logic

Setting Up SSH and Running Playbooks

```

# Install Ansible on WSL (Ubuntu)
sudo apt update
sudo apt install -y ansible

# Navigate to project root (from WSL, Windows drives are under /mnt/)
cd /mnt/d/Coding/devops/"Final Part - 2026"

# Generate a dedicated SSH key for Ansible automation
ssh-keygen -t ed25519 -a 64 -f ~/.ssh/id_ed25519 -C "devops-course-ansible"
# -t ed25519: Modern, secure elliptic curve algorithm

```

```
# -a 64: 64 rounds of key derivation (makes brute force harder)
# -C: Comment/label for the key

# Copy the public key to the VPS (authorizes passwordless login)
ssh-copy-id -i ~/.ssh/id_ed25519.pub root@187.127.157.4

# Verify SSH works manually
ssh -i ~/.ssh/id_ed25519 root@187.127.157.4

# Set ANSIBLE_CONFIG so it finds your ansible.cfg in the project
directory
export ANSIBLE_CONFIG="$PWD/ansible.cfg"

# Verify Ansible configuration
ansible --version | sed -n '1,6p'

# Test connectivity (ping all hosts in "prod" group)
ansible prod -m ping

# Run the bootstrap playbook (set up Docker, Git, etc.)
ansible-playbook bootstrap.yml

# Run the deployment playbook (pull and start the app)
ansible-playbook deploy.yml

# Run the observability playbook (deploy monitoring stack)
ansible-playbook observability.yml
```

Key `ansible.cfg` settings:

```
[defaults]
inventory = inventory/hosts
remote_user = root
private_key_file = ~/.ssh/id_ed25519
host_key_checking = False    # Skip fingerprint prompt (safe for
known hosts)

[ssh_connection]
ssh_args = -o StrictHostKeyChecking=no
```

The 4-Step Ansible Deployment Flow

Your Laptop (Control Node)

↓

Inventory → Select "prod" group

↓

1. bootstrap.yml → make server ready
(install Docker, Docker Compose, Git, create app directories)

↓

2. deploy.yml → app is deployed
(git pull, docker compose up -d --build)

↓

3. verify → health endpoint
(curl http://VPS_IP/api/health)

↓

VPS (Managed Node) – app is running

5.4 — Caddy: Reverse Proxy with Automatic HTTPS

What & Why

What Caddy is: A modern web server and reverse proxy designed to be simple to configure while handling one of the most painful parts of web infrastructure automatically: **SSL/TLS certificates**.

The Old Way (The Problem): Before tools like Caddy, setting up HTTPS required:

1. Manually requesting a certificate from Let's Encrypt using Certbot
2. Writing complex Nginx/Apache configuration to handle HTTP→HTTPS redirects
3. Setting up a cron job to auto-renew certificates every 90 days
4. Debugging cryptic OpenSSL errors when renewals failed

What Caddy Does Differently: Caddy handles **all of this automatically**. You give it a domain name, and it:

- Automatically requests an SSL certificate from Let's Encrypt
- Proves domain ownership via HTTP challenge
- Installs and configures HTTPS
- Renews certificates before they expire — silently, in the background


```
# Status page subdomain (Uptime Kuma)
statusdevopscourseankit.duckdns.org {
    reverse_proxy uptime-kuma:3001
}
```

Caddy automatically handles HTTPS for both domains — no extra configuration needed.

Docker Compose Integration

```
# docker-compose.yml (production, on the VPS)
services:
  caddy:
    image: caddy:latest
    container_name: caddy
    restart: unless-stopped
    ports:
      - "80:80"      # HTTP (also used for ACME challenge / Let's
Encrypt verification)
      - "443:443"    # HTTPS
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro      # Mount
configuration
      - caddy_data:/data                          # Persist
certificates (critical!)
      - caddy_config:/config                      # Persist config
cache
    networks:
      - app-net

volumes:
  caddy_data:    # Stores issued TLS certificates – must persist
across restarts!
  caddy_config:
```

Why persist `caddy_data` ? This volume stores the SSL certificates Caddy obtains from Let's Encrypt. If you lose this volume, Caddy must re-request certificates on the next restart. Let's Encrypt has rate limits (5 certificates per domain per week), so losing certs frequently can lock you out temporarily.

How Caddy Handles TLS Certificates (Step by Step)

1. Caddy starts and reads the Caddyfile — sees
`devopscourseankit.duckdns.org`
 2. Caddy checks its cert store — no certificate exists yet
 3. Caddy contacts Let's Encrypt: "I need a certificate for
`devopscourseankit.duckdns.org` "
 4. Let's Encrypt sends a challenge: "Prove you control that domain"
 5. Caddy hosts a temporary file at
`http://devopscourseankit.duckdns.org/.well-known/acme-challenge/...`
 6. Let's Encrypt fetches that file — domain ownership verified
 7. Let's Encrypt issues the certificate — Caddy installs it automatically
 8. Caddy serves all traffic over HTTPS immediately
 9. ~30 days before expiry, Caddy silently renews the certificate in the background
-

Troubleshooting Caddy TLS Issues

If the HTTPS domain isn't working after deployment:

```
# SSH into the VPS
ssh root@187.127.157.4
cd /opt/statusboard

# Restart Caddy so it retries TLS cert provisioning
docker compose restart caddy

# Watch the logs to confirm cert is being issued
docker compose logs -f caddy

# If Caddy logs show it's stuck (failed ACME attempts cached):
# Clear old/failed cert cache and start fresh
docker compose down
docker volume rm statusboard_caddy_data
docker compose up -d

# Caddy will request fresh certificates on startup

# Using Ansible to trigger a full redeploy remotely:
```

```
ansible prod -b -m shell -a "cd /opt/statusboard && docker compose
-f compose.yaml down && docker compose -f compose.yaml up -d --
build"
```

DuckDNS — Getting a Free Domain: DuckDNS is a free dynamic DNS service that gives you a subdomain like `yourname.duckdns.org` and lets you point it to any IP address. In this course, it's used to get a real domain for testing HTTPS without buying a domain.

```
https://devopscourseankit.duckdns.org      ← Main app
https://statusdevopscourseankit.duckdns.org ← Uptime Kuma status
page
```

Both point to the same VPS IP via DuckDNS.

5.5 — Monitoring Architecture Overview

Before diving into each tool, understand what "monitoring" actually means in production. It is split into three distinct concerns:

1. **Uptime Monitoring (Uptime Kuma):** Is the service up or down? Binary health checks from outside the system.
2. **Metrics Monitoring (Prometheus + Grafana):** *Why* is it slow or failing? Numeric data: CPU%, RAM usage, request latency, container resource consumption.
3. **Application Observability (Sublyzer):** What errors are users experiencing? Real-time error tracking, performance traces, and AI-assisted debugging.

The Three Pillars of Observability (DevOps Standard):

Pillar	What It Captures	Tools
Metrics	Numeric values over time (CPU %, disk usage, request count)	Prometheus, Grafana
Logs	Detailed text records of discrete events	Loki, ELK Stack (Elasticsearch/Logstash/Kibana)
Traces	The path of a single request through distributed services	Jaeger (pronounced "yagger"), Grafana Tempo

In this course, the focus is on **Metrics** (Prometheus/Grafana) and **Uptime** (Uptime Kuma), with Sublyzer handling application-level observability.

5.6 — Uptime Kuma: Availability Monitoring

What & Why

What Uptime Kuma is: A self-hosted monitoring tool that repeatedly sends HTTP/HTTPS requests to your application endpoints at regular intervals (e.g., every 60 seconds) to determine if they are up or down. It is the "first line of defense" — it tells you immediately when your service becomes unreachable.

The Difference from Prometheus: Uptime Kuma is **synthetic monitoring** — it only knows if an endpoint returns a successful response code from the *outside*. It cannot tell you *why* it failed (was the CPU maxed out? Did the database crash?). For the "why," you need Prometheus.

Think of it this way: Uptime Kuma is the burglar alarm that tells you the front door is open. Prometheus + Grafana is the security camera system that shows you who opened it and how.

Key Capabilities

- **Up/Down Status:** Real-time visibility into endpoint availability, updated after each check interval
 - **Response Time Graphs:** Visual tracking of latency over time — spikes indicate performance degradation
 - **Incident History:** Full record of past outages with timestamps and durations — critical for SLA reporting
 - **Alerting:** Push notifications via Slack, Email, Discord, Telegram, and more — configurable per monitor
 - **Status Pages:** Public-facing pages you can share with users or stakeholders showing service health (e.g., `statusdevopscourseankit.duckdns.org`)
 - **Certificate Expiry:** Shows when your SSL certificate expires (from the Uptime Kuma dashboard screenshot, cert expiry shows "90 days")
-

Setting Up Uptime Kuma in Docker Compose

```
# In docker-compose.yml (statusboard project)
services:
  uptime-kuma:
    image: louislam/uptime-kuma:latest
    container_name: uptime-kuma
    restart: unless-stopped
    volumes:
      - uptime-kuma-data:/app/data      # Persist monitor configs and
history
    networks:
      - app-net

volumes:
  uptime-kuma-data:
```

Caddy routes the status subdomain to Uptime Kuma:

```
statusdevopscourseankit.duckdns.org → Caddy → uptime-kuma:3001
```

Uptime Kuma Dashboard in Action

After deployment and setup:

1. Navigate to `https://statusdevopscourseankit.duckdns.org`
2. Create an admin account with a strong password
3. Add monitors: **New Monitor** → HTTP(s) → Enter your endpoint URL → Set check interval
4. Add monitors for:
 - `https://devopscourseankit.duckdns.org` (frontend)
 - `https://devopscourseankit.duckdns.org/api/health` (backend API)

The dashboard shows:

- **Response time graph** — latency trend over the last 24 hours
- **Uptime percentage** — 24-hour and 30-day averages
- **Incident log** — timestamped list of up/down events with HTTP status codes

Homework exercise: Stop and restart a Docker container while watching the Uptime Kuma dashboard — observe how it detects the outage, marks it as "Down" with a timestamp, and then recovers when the container restarts.

5.7 — Prometheus & Grafana: Metrics Collection and Visualization

What & Why

The Gap Uptime Kuma Leaves: Uptime Kuma tells you *that* your server is slow or down. It cannot tell you *why* — was it a CPU spike? RAM exhaustion? A misbehaving container consuming all resources?

Prometheus and Grafana fill this gap:

- **Prometheus:** Acts as the **metrics database and collector**. It stores time-series data — every metric is a numeric value associated with a specific timestamp and a set of labels. It operates on a **pull model** — every 15 seconds, it actively reaches out to its configured targets to scrape the latest metrics.
- **Grafana:** The **visualization and dashboarding layer**. It does NOT store any data. It connects to Prometheus as a data source, executes PromQL queries, and renders the results as graphs, charts, stat panels, and tables.

Key architectural clarification:

```
Node Exporter + cAdvisor + Blackbox → expose metrics endpoints
    ↓ (Prometheus pulls/scrapes every 15s)
Prometheus (time-series database)
    ↓ (Grafana queries via PromQL)
Grafana (visualization - renders dashboards)
```

The Exporter / Adapter Concept

Prometheus relies on **exporters** to collect metrics from different systems. An exporter is a small program that runs alongside the thing being monitored and exposes metrics in a format Prometheus can scrape.

Exporter	What It Monitors	Port	Type
Node Exporter	Host-level: CPU, RAM, disk I/O, network traffic	9100	White Box (inside)
cAdvisor	Container-level: per-container CPU, memory, network	8080	White Box (inside)
Blackbox Exporter	External probing: HTTP/HTTPS endpoints, DNS, TCP	9115	Black Box (outside)

Node Exporter vs. cAdvisor — Why You Need Both:

- **Node Exporter** tells you "the host CPU is at 90%." This is the big picture — the server is struggling.
- **cAdvisor** tells you "the backend container is consuming 85% of the CPU." This is the drill-down — now you know which container is the culprit.

If your Node Exporter reports 90% CPU, you look at cAdvisor to find which specific container is responsible. Both are essential for complete diagnostics.

White Box vs. Black Box Monitoring

White Box Monitoring: Observing *internal* metrics from inside the system. Relies on Node Exporter (host metrics) and cAdvisor (container metrics).

- What it catches: CPU spikes, memory leaks, container crashes, disk filling up
- What it misses: DNS failures, issues with reverse proxies, network routing problems

Black Box Monitoring: Treating the system as a black box and probing it from the *outside*, exactly as a user would. The Blackbox Exporter sends HTTP/HTTPS requests to your public URLs and reports success/failure and response time.

- What it catches: DNS failures, routing problems, certificate errors, issues that only manifest externally
- What it misses: Internal resource bottlenecks (a request might succeed but internal CPU is maxed)

You need both: Black box tells you the door is broken from the user's perspective. White box tells you why the door is broken internally. Together, they

give complete observability.

Prometheus Configuration (prometheus.yml)

```
global:
  scrape_interval: 15s      # How often Prometheus polls each target
  evaluation_interval: 15s # How often alerting rules are evaluated

scrape_configs:
  # Prometheus monitors itself (self-monitoring loop)
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]
        labels:
          instance: "prometheus"

  # Node Exporter – host hardware and OS metrics
  - job_name: "node_exporter"
    static_configs:
      - targets: ["node-exporter:9100"]
        labels:
          instance: "node_exporter"

  # cAdvisor – per-container metrics
  - job_name: "cadvisor"
    static_configs:
      - targets: ["cadvisor:8080"]
        labels:
          instance: "cadvisor"

  # Blackbox Exporter – external HTTP probes
  - job_name: "blackbox_http"
    metrics_path: /probe
    params:
      module: [http_2xx] # Use the http_2xx module
                        # defined in blackbox.yml
    static_configs:
      - targets:
          - "https://devopscourseankit.duckdns.org"
          - "https://devopscourseankit.duckdns.org/api/health"
        labels:
          job: "blackbox_http"
```

```

relabel_configs:
  - source_labels: [__address__]
    target_label: __param_target
  - source_labels: [__param_target]
    target_label: instance
  - target_label: __address__
    replacement: blackbox:9115 # Blackbox Exporter address
within Docker network

```

Why pass the URL in `prometheus.yml` for Blackbox? Node Exporter and cAdvisor live *inside* the system — they just expose metrics at their ports, and Prometheus scrapes those ports. The Blackbox Exporter is different: it is an external probe that needs to be told **which URLs to probe**. By passing the URLs in `prometheus.yml`, you are telling Prometheus: "Use the Blackbox Exporter to perform external health checks against these specific destinations."

Blackbox Exporter Configuration (`blackbox.yml`)

```

modules:
  http_2xx:
    prober: http # Use HTTP prober
    timeout: 5s
    http:
      valid_http_versions: ["HTTP/1.1", "HTTP/2.0"]
      valid_status_codes: [200, 201, 204] # What counts as
"success"
      method: GET
      follow_redirects: true
      tls_config:
        insecure_skip_verify: false # Validate SSL certs
properly

```

Time-Series Data Format (How Prometheus Stores Data)

Every metric Prometheus stores consists of three components:

Metric Name	Labels (key=value pairs)	Value
Timestamp		

```
node_cpu_seconds_total{mode="idle", instance="node_exporter"}
1234567.89      1716000000
probe_success{instance="https://devopscourseankit.duckdns.org",
job="blackbox_http"}      1      1716000015
container_memory_usage_bytes{name="backend", job="cadvisor"}
134217728      1716000015
```

Labels are what make Prometheus powerful — they let you slice and dice data. Query all containers with `container_memory_usage_bytes`, or drill down to just the backend: `container_memory_usage_bytes{name="backend"}`.

Docker Compose for the Observability Stack

```
# observability/compose.yaml
services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    restart: unless-stopped
    ports:
      - "9090:9090"          # Prometheus web UI
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro      #
Config
      - prometheus_data:/prometheus                              #
Time-series database
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.retention.time=30d'    # Keep 30 days of
data
    networks:
      - monitoring

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    restart: unless-stopped
    ports:
      - "3000:3000"          # Grafana web UI
```

```
volumes:
  - grafana_data:/var/lib/grafana    # Persists dashboards and
settings
environment:
  - GF_SECURITY_ADMIN_USER=admin
  - GF_SECURITY_ADMIN_PASSWORD=your_secure_password
networks:
  - monitoring
```

```
node-exporter:
  image: prom/node-exporter:latest
  container_name: node-exporter
  restart: unless-stopped
  ports:
    - "9100:9100"
  volumes:
    - /proc:/host/proc:ro    # Read-only access to host's /proc
filesystem
    - /sys:/host/sys:ro      # Read-only access to host's /sys
filesystem
    - /:/rootfs:ro
  command:
    - '--path.procfs=/host/proc'
    - '--path.sysfs=/host/sys'
  networks:
    - monitoring
```

```
cadvisor:
  image: gcr.io/cadvisor/cadvisor:latest
  container_name: cadvisor
  restart: unless-stopped
  ports:
    - "8080:8080"
  volumes:
    - /:/rootfs:ro
    - /var/run:/var/run:ro
    - /sys:/sys:ro
    - /var/lib/docker/:/var/lib/docker:ro
  networks:
    - monitoring
```

```
blackbox:
  image: prom/blackbox-exporter:latest
  container_name: blackbox
```

```

restart: unless-stopped
ports:
  - "9115:9115"
volumes:
  - ./blackbox.yml:/config/blackbox.yml:ro
command:
  - '--config.file=/config/blackbox.yml'
networks:
  - monitoring

volumes:
  prometheus_data:
  grafana_data:

networks:
  monitoring:
    driver: bridge

```

Why volumes for Prometheus and Grafana?

- **Prometheus data:** Without a volume, every container restart wipes all collected metrics — you lose all historical trends.
- **Grafana data:** Without a volume, all your custom dashboards and data source configurations are lost on restart.

Why a shared network? Within the shared `monitoring` Docker network, containers reference each other by service name (e.g., `prometheus:9090`, `cadvisor:8080`). Grafana uses `prometheus:9090` as the data source URL — this only works because both are on the same network.

Deploying the Observability Stack with Ansible

```

# observability.yml Ansible playbook
---
- name: Deploy observability stack
  hosts: prod
  become: true

  tasks:
    - name: Create observability directory on VPS
      file:

```

```
    path: /opt/observability
    state: directory
    mode: '0755'

- name: Copy docker-compose file
  copy:
    src: ../observability/compose.yaml
    dest: /opt/observability/compose.yaml

- name: Copy prometheus config
  copy:
    src: ../observability/prometheus.yml
    dest: /opt/observability/prometheus.yml

- name: Copy blackbox config
  copy:
    src: ../observability/blackbox.yml
    dest: /opt/observability/blackbox.yml

- name: Start observability stack
  shell: |
    cd /opt/observability
    docker compose up -d
```

```
# Deploy
ansible-playbook observability.yml

# Access
# Prometheus: http://187.127.157.4:9090/targets
# Grafana:    http://187.127.157.4:3000/
```

Verifying the Stack Works: PromQL Queries

After deployment, verify everything is working in the Prometheus UI (/targets page — all targets should show UP state).

Essential PromQL verification queries:

```
# Are all scrape targets healthy? (1 = up, 0 = down)
up
```

```
# Are external probes succeeding? (1 = success, 0 = failure)
probe_success

# How long are probes taking? (response latency)
probe_duration_seconds

# What HTTP status code did the probe receive?
probe_http_status_code

# Host CPU usage across all cores
100 - (avg by(instance) (irate(node_cpu_seconds_total{mode="idle"}
[5m]))) * 100

# Total RAM used
node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes
```

In Prometheus UI (http://VPS_IP:9090/targets): The Targets page shows all configured scrape jobs with their state (UP / DOWN), last scrape time, and scrape duration. This is the primary verification step.

From the course: the Targets page showed:

- `blackbox_http` (2/2 up) — both HTTP probes working
- `cadvisor` (1/1 up) — container metrics being collected
- `node_exporter` (1/1 up) — host metrics being collected
- `prometheus` (1/1 up) — self-monitoring loop active

Setting Up Grafana Dashboards

1. Access Grafana at http://VPS_IP:3000 → log in with admin credentials
2. **Add Data Source:** Connections → Data Sources → Add → Prometheus
 - URL: <http://prometheus:9090> (container name, not IP — same Docker network)
 - Click "Save & Test" — should show "Data source is working"
3. **Create a Dashboard:** Dashboards → New Dashboard → Add Panel
4. **Run a PromQL query:** In the Query box, type `probe_success` → click "Run queries"
5. The panel shows a time-series graph of probe success (values of 1 = up, 0 = down)

6. Add a second panel with `probe_duration_seconds` to track latency over time

Crucial architectural point: Grafana does NOT store metrics. It is purely a visualization layer. Every time you load a dashboard, Grafana sends a PromQL query to the Prometheus API and renders the response. The data lives in Prometheus's time-series database.

The Complete Observability Data Flow

1. Infrastructure Level (Adapters expose metrics)
Node Exporter → exposes host metrics at `:9100/metrics`
cAdvisor → exposes container metrics at `:8080/metrics`
2. External Level (Blackbox probes your public URLs)
Blackbox Exporter → checks `https://devopscourseankit.duckdns.org` every 15s
3. Collection (Prometheus pulls from all sources)
Prometheus → scrapes all targets every 15s → stores as time-series data
4. Storage (Prometheus time-series database)
All metrics stored as: `{metric_name, labels, value, timestamp}`
Retained for 30 days (configurable)
5. Visualization (Grafana queries and renders)
Grafana → connects to Prometheus via `"prometheus:9090"`
→ executes PromQL queries → renders results as dashboards
→ makes patterns and anomalies human-readable

5.8 — Grafana k6: Load Testing

What & Why

What k6 is: A scriptable load generator that lets you write test logic in **JavaScript** to simulate real-world concurrent user traffic against your deployed application. It measures how your system behaves under pressure.

Why load testing matters: An application might work perfectly for 1 user but crash for 1,000 simultaneous users. Load testing reveals:

- **Latency degradation** — Does response time increase as traffic increases?
- **System stability** — Does the app crash or return errors under stress?
- **Resource behavior** — How does CPU/RAM respond to increasing load?
(Visible in Grafana while k6 runs)

Key metrics k6 reports:

- **P50, P95, P99 latencies** — "95% of requests completed within X milliseconds." P99 is the most important for understanding tail latency (the worst-case experience for users).
 - **Error rate** — What percentage of requests failed?
 - **Request throughput** — How many requests per second was the system handling?
-

Writing a k6 Test

```
// ops/tests/k6/statusboard.js
import http from 'k6/http';
import { check, sleep } from 'k6';

// Test configuration: traffic ramp-up stages
export const options = {
  stages: [
    { duration: '15s', target: 5 }, // Ramp up to 5 virtual
    users over 15 seconds
    { duration: '30s', target: 15 }, // Ramp up to 15 virtual
    users over 30 seconds
    { duration: '15s', target: 0 }, // Ramp down to 0 (cool-
    down)
  ],
};

const BASE_URL = __ENV.BASE_URL ||
  'https://devopscourseankit.duckdns.org';

export default function () {
  // Test 1: Health check endpoint
```

```

const healthRes = http.get(`${BASE_URL}/api/health`);
check(healthRes, {
  'health check status is 200': (r) => r.status === 200,
});

// Test 2: Main page
const webRes = http.get(`${BASE_URL}/`);
check(webRes, {
  'web page status is 200': (r) => r.status === 200,
});

sleep(1); // Wait 1 second between iterations (simulates real
user behavior)
}

```

Stages explained: The ramp-up approach (starting small and increasing) is critical. Hitting a server with 100 simultaneous users instantly can cause a spike that doesn't reflect real usage patterns. Gradual ramp-up reveals at exactly what traffic level performance starts degrading.

Running k6 from the VPS

k6 is run directly on the VPS using Docker, so the test traffic originates from the same server (or you can run it remotely):

```

# SSH into the VPS
ssh root@187.127.157.4
cd /opt/statusboard

# Run the load test using the k6 Docker image
docker run --rm -i \
  -e BASE_URL="https://devopscourseankit.duckdns.org" \
  -v "$PWD:/work" \
  -w /work \
  grafana/k6 run ops/tests/k6/statusboard.js

```

Deploying the test file via Ansible (deploy.yml additions):

```

- name: Ensure k6 directory exists on VPS
  file:

```

```
path: /opt/statusboard/ops/tests/k6
state: directory
```

– name: Sync k6 tests to VPS

copy:

src: ops/tests/k6/

dest: /opt/statusboard/ops/tests/k6/

Interpreting k6 Output

```
      /\      Grafana  /--/
     /\  /  \   | \  _  /  /
    /  \ /    \  |  / /  /  --\
   /      \    |  (  |  (  )  |
  /  ----- \  |  | \ \  \ ----/
```

execution: local

script: ops/tests/k6/statusboard.js

output: -

scenarios: (100.00%) 1 scenario, 15 max VUs, 1m30s max duration

* default: Up to 15 looping VUs for 1m0s over 3 stages

✓ health check status is 200

✓ web page status is 200

checks.....: 100.00% ✓ 458 ✗ 0

data_received.....: 2.8 MB 46 kB/s

data_sent.....: 41 kB 677 B/s

http_req_duration.....: avg=207ms min=8ms med=12ms
max=1.2s p(90)=890ms p(95)=1.1s

http_reqs.....: 229 3.8/s

vus.....: 1 min=1 max=15

vus_max.....: 15 min=15 max=15

Reading the results:

- checks 100.00% — All requests returned the expected status codes (no errors)

- `http_req_duration p(95)=1.1s` — 95% of requests completed within 1.1 seconds. If your SLA requires sub-500ms responses, this tells you the system is struggling under 15 concurrent users.
- `http_reqs 3.8/s` — The system handled 3.8 requests per second during the test

Watching Grafana simultaneously: While k6 runs, open your Grafana dashboard showing CPU usage. You will see CPU spike visibly as virtual users ramp up — this is the resource behavior k6 is designed to expose.

5.9 — Sublyzer: Application Observability

What Sublyzer Is

While Prometheus/Grafana monitors **infrastructure-level health** (servers, containers, CPU, RAM) and Uptime Kuma monitors **availability**, Sublyzer provides **application-level observability** — tracking what users are actually experiencing inside the app itself: errors they encounter, slow page loads, JavaScript exceptions, and API failures.

Think of it as a "dashboard" installed inside your app:

- Shows who is using the app right now (active users)
 - Tracks how fast the app is responding to each user
 - Captures and categorizes every error or crash a user experiences
 - Provides AI-assisted analysis to help diagnose complex bugs quickly
-

Setup Steps

1. **Create a project** on the Sublyzer platform and get your unique integration code (the "secret password" that connects your app to Sublyzer's servers)
2. **Add the SDK** to your project's public folder and paste the integration code to activate monitoring
3. **View the dashboard** — once the app is running, the Sublyzer dashboard immediately shows:
 - **Active Users** — real-time count of who is using the app
 - **Performance score** — average response time score (0-100)

- **System Health score** — composite health score based on errors, vulnerabilities, and load time
 - **Error tracking** — under the Security tab, every crash or bug is captured with its stack trace, severity (critical/high/medium/low), and timestamp
4. **AI Solutions** — Connect your GitHub repository to enable AI-assisted bug fixing. Sublyzer can analyze errors and suggest code changes based on the actual source code.

Where Sublyzer Fits in the Monitoring Stack

Tool	Answers
Uptime Kuma	"Is the server reachable?"
Prometheus + Node Exporter	"Is the host CPU/RAM healthy?"
Prometheus + cAdvisor	"Which container is consuming excessive resources?"
Prometheus + Blackbox	"Are the public HTTPS endpoints responding?"
Grafana	"What do all these metrics look like visually over time?"
Sublyzer	"What errors are real users experiencing inside the app right now?"
k6	"How does the system behave when we simulate heavy traffic?"

Together, these tools provide complete observability from the infrastructure layer down to the individual user experience.

5.10 — Phase 5 Summary: The Complete Production Flow

```
Step 1: Write Terraform config (.tf files)
terraform init → terraform fmt → terraform validate → terraform
plan → terraform apply
VPS is now provisioned with a public IP (e.g., 187.127.157.4)
```

↓

Step 2: Write Ansible playbooks

ansible-playbook bootstrap.yml → Docker, Git, dependencies installed on VPS

ansible-playbook deploy.yml → App cloned, .env configured, docker compose up -d

↓

Step 3: Caddy handles routing and HTTPS

Caddyfile maps domains to containers → Auto-requests Let's Encrypt certificates

devopscourseankit.duckdns.org → accessible over HTTPS

↓

Step 4: Ansible deploys monitoring stack

ansible-playbook observability.yml → Prometheus, Grafana, exporters running

Uptime Kuma → monitors endpoint availability

↓

Step 5: Full observability active

Prometheus scrapes exporters every 15s → time-series data stored

Grafana connects to Prometheus → dashboards visualize system health

k6 runs load tests → validate system under concurrent user traffic

Sublyzer tracks application errors → AI-assisted debugging

Interview Edge: Phase 5 Questions

Q: "What is the difference between Provisioning and Configuration

Management?" A: Provisioning (Terraform) is about creating and managing the existence of infrastructure — servers, networks, DNS. It talks to cloud provider APIs and maintains state in `terraform.tfstate`. Configuration management (Ansible) is about making that infrastructure ready to use — installing software, configuring the OS, deploying applications. It connects over SSH and executes tasks on the server. Terraform outputs (like VPS IP) feed into Ansible inputs (inventory).

Q: "Why is `terraform.tfstate` stored remotely in a team environment?" A:

Two reasons. First, if two engineers run `terraform apply` simultaneously with local state files, their states diverge — a "split brain" scenario where each thinks they own different resources, leading to destructive conflicts. Second, the state file contains sensitive data (resource IDs, sometimes passwords) that should not sit on an unencrypted laptop. Remote backends like AWS S3 + DynamoDB provide centralized, locked, secure state storage.

Q: "What is Ansible's idempotency and why does it matter?" A: Idempotency means running the same Ansible playbook multiple times produces the same result without side effects. If Docker is already installed, the `apt` module returns `ok` (skips installation). If a config file is already correct, the `copy` module returns `ok`. This makes Ansible safe to run in CI/CD pipelines and as part of automated deployments — you can rerun the playbook on every deploy without worrying about double-installing packages or overwriting valid configurations.

Q: "What is the difference between Prometheus and Grafana?" A: Prometheus is the **data collection and storage layer** — it scrapes metrics from exporters every 15 seconds and stores them as time-series data in its own database. Grafana is the **visualization layer** — it does not store any data. It connects to Prometheus as a data source, executes PromQL queries, and renders the results as human-readable dashboards. Grafana without Prometheus has no data; Prometheus without Grafana has data but no visual interface.

Q: "Why do you need both white-box and black-box monitoring?" A: Black-box monitoring (Blackbox Exporter) checks your system from the outside — exactly as a user would experience it. It catches DNS failures, routing issues, and problems that only manifest externally. White-box monitoring (Node Exporter, cAdvisor) provides internal visibility — CPU saturation, memory leaks, container-level resource consumption. A service can pass all black-box checks (HTTP 200, fast response) while internally the CPU is at 99% heading toward failure. Both perspectives together provide complete system understanding.

Q: "Why use Caddy over Nginx for production HTTPS?" A: Caddy handles SSL/TLS certificate management **completely automatically** — it requests certificates from Let's Encrypt, proves domain ownership, and renews them before expiry without any manual intervention. With Nginx, you must manually configure Certbot, set up cron jobs for renewals, and debug renewal failures. For a small team or personal project, Caddy dramatically reduces operational overhead while providing enterprise-grade HTTPS.